



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES
LICENCIATURA EN INGENIERIA DE SOFTWARE



Materia: INGENIERIA WEB

Profesora: IRINA FONG

PROYECTO FINAL
SISTEMA DE MATRICULA

Integrantes:
Jesse, Kazim. – 8-988-1702
Carrión, Juan. - 8-994-358
Núñez, Cristopher. – 8-977-340

GRUPO: 1SF-131

12/15/2025

Indice

Introducción	3
1. Requerimientos Funcionales.....	4
2. Requerimientos No Funcionales	4
3. Actores del Sistema.....	5
Arquitectura	6
Diseño UML.....	7
1. Diagrama General de Casos de Uso	7
2. Casos de Uso Específicos.....	8
3. Diagramas de secuencia.....	12
4. Diagramas de actividad	19
5. Diagramas de estado.....	28
6. Diagramas de clase	35
7. Diagramas de componente	42
Base de datos	50
Diseño del sistema	51
Repositorio y video	52
Conclusión	53
Bibliografía	54

Introducción

Dentro de este proyecto sobre el sistema de matrícula se busca que tenga como finalidad gestionar de forma eficiente el proceso académico de inscripción de estudiantes en distintas materias. Así de esta forma el sistema centraliza la administración de usuarios, estudiantes, docentes, planes de estudio, grupos, horarios y el proceso de matrícula, permitiendo que la institución controle su información de forma organizada y automatizada. Se utilizaron tecnologías web junto con una base de datos estructurada y un diseño modular basado en UML para garantizar claridad y escalabilidad.

El objetivo general

Se busca desarrollar un sistema web de matrícula que permita gestionar de forma eficiente, segura y organizada el proceso de inscripción de los estudiantes, aplicando validaciones académicas y administrativas propias de una institución educativa.

Los objetivos específicos

- Automatizar el proceso de matrícula de estudiantes.
- Gestionar estudiantes, docentes, materias, horarios y grupos.
- Aplicar validaciones de prerrequisitos, cupos y choques de horario.
- Garantizar la integridad y seguridad de la información académica.

El alcance del sistema

El sistema de matrícula busca cubrir la gestión académica desde la administración de usuarios hasta el proceso de inscripción del estudiante en las materias disponibles, permitiendo consultas de horarios, control de cupos y generación de comprobantes de matrícula.

Requerimientos del Sistema

1. Requerimientos Funcionales

- Gestionar estudiantes
- Gestionar docentes
- Gestionar usuarios y roles
- Gestionar materias con códigos únicos y prerequisites
- Gestionar plan de estudios
- Gestionar horarios
- Gestionar grupos
- Realizar matrícula con validaciones
- Consultar horarios
- Generar comprobante de matrícula

2. Requerimientos No Funcionales

- Usabilidad: interfaz clara y simple
- Rendimiento: consultas optimizadas y validaciones rápidas
- Seguridad: manejo de roles y autenticación
- Confiabilidad: operaciones de matrícula con transacciones

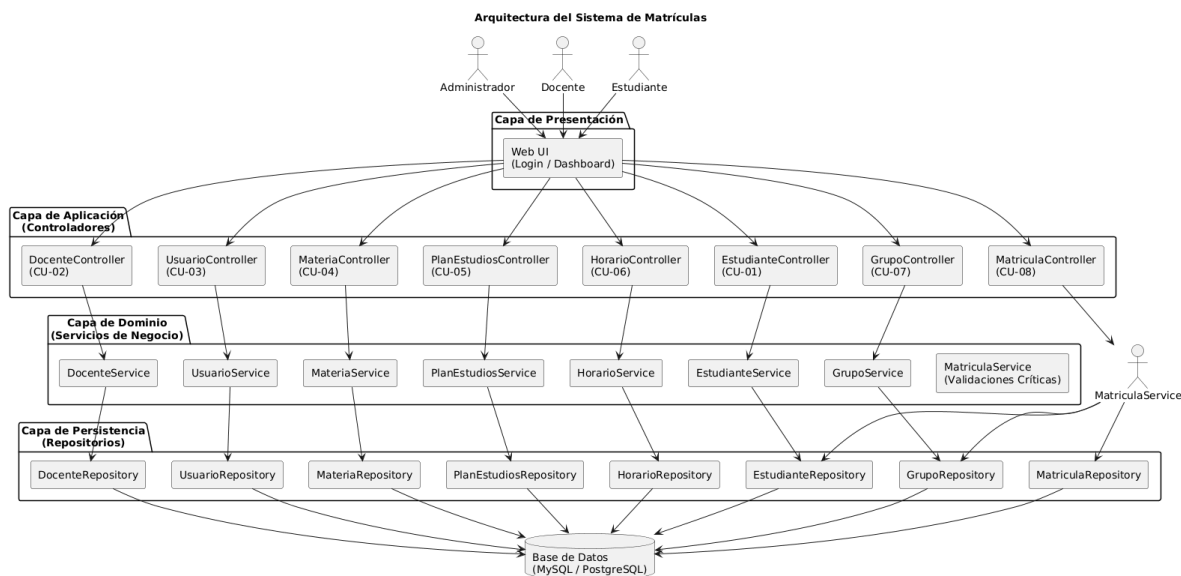
- Mantenibilidad: arquitectura modular basada en clases
- Integridad: validación de datos y restricciones de BD

3. Actores del Sistema

Según el modelado funcional:

- Administrador: controla todos los módulos del sistema
- Docente: consulta horarios y grupos asignados
- Estudiante: revisa materias disponibles, ve horarios y realiza matrícula

Arquitectura



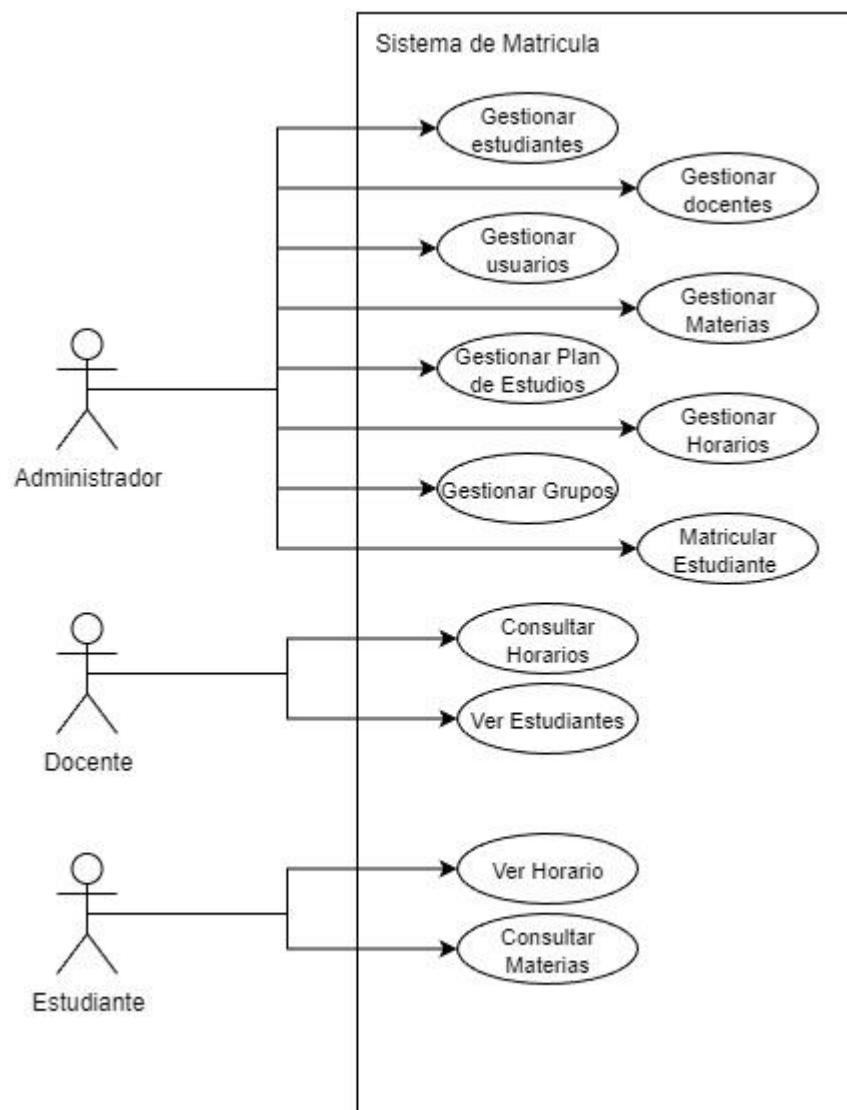
Para el desarrollo del sistema de matrículas se propone una **arquitectura de software en capas (3-tier)**, implementada como un **monolito modular basado en el patrón MVC de Laravel**. La solución se organiza en una **capa de presentación**, donde se gestionan las vistas Blade y los controladores web; una **capa de aplicación o lógica de negocio**, compuesta principalmente por servicios como MatriculaService y AuthenticationService, que encapsulan los casos de uso del sistema; y una **capa de acceso a datos**, formada por modelos Eloquent, repositorios e interfaces que permiten desacoplar la lógica de negocio de la persistencia.

Esta arquitectura facilita la separación de responsabilidades, mejora la mantenibilidad del código y permite que el sistema pueda escalar funcionalmente en el futuro, por ejemplo añadiendo módulos de pagos, reportes o gestión de horarios, sin necesidad de cambiar el estilo arquitectónico base.

Diseño UML

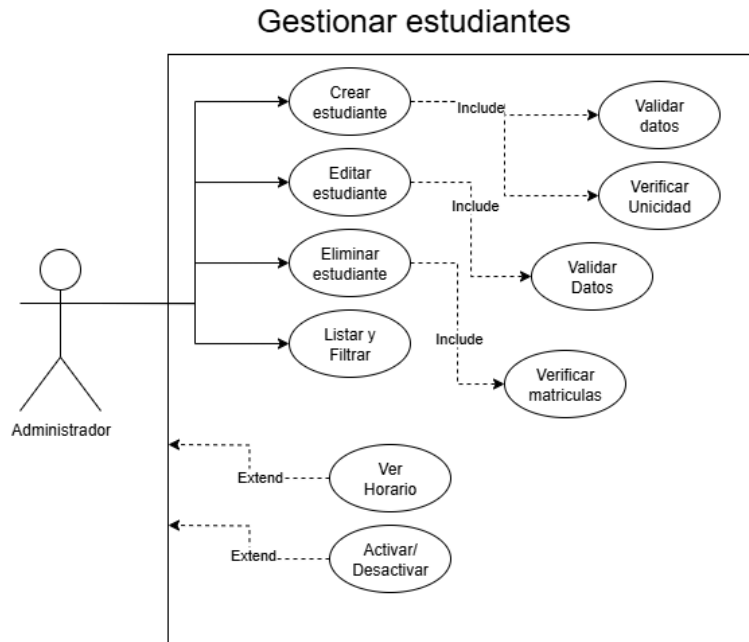
1. Diagrama General de Casos de Uso

“Este diagrama de casos de uso representa la interacción entre los actores del sistema y las principales funcionalidades del sistema de matrícula, permitiendo identificar las responsabilidades del administrador, docente y estudiante.”

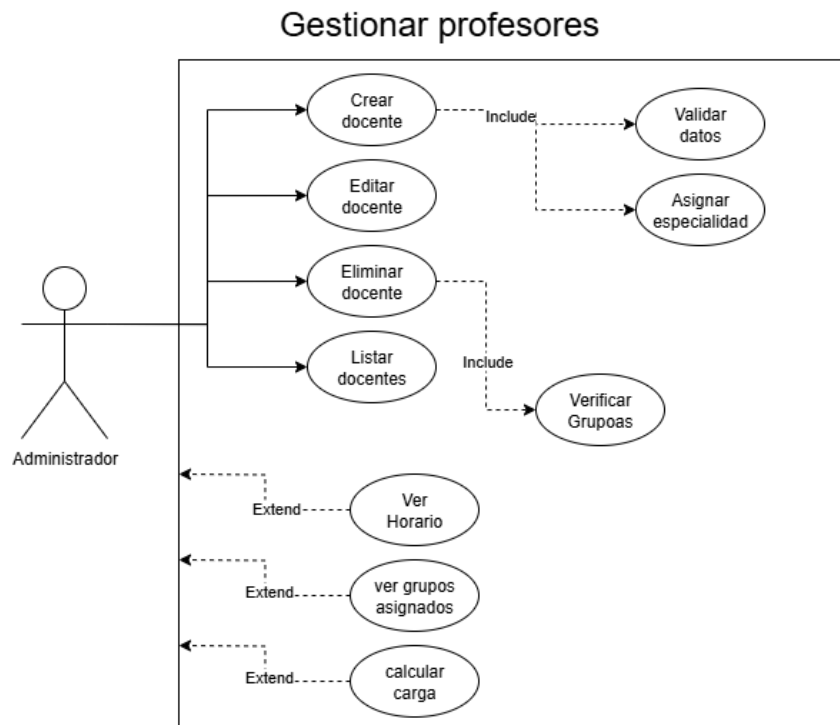


2. Casos de Uso Específicos

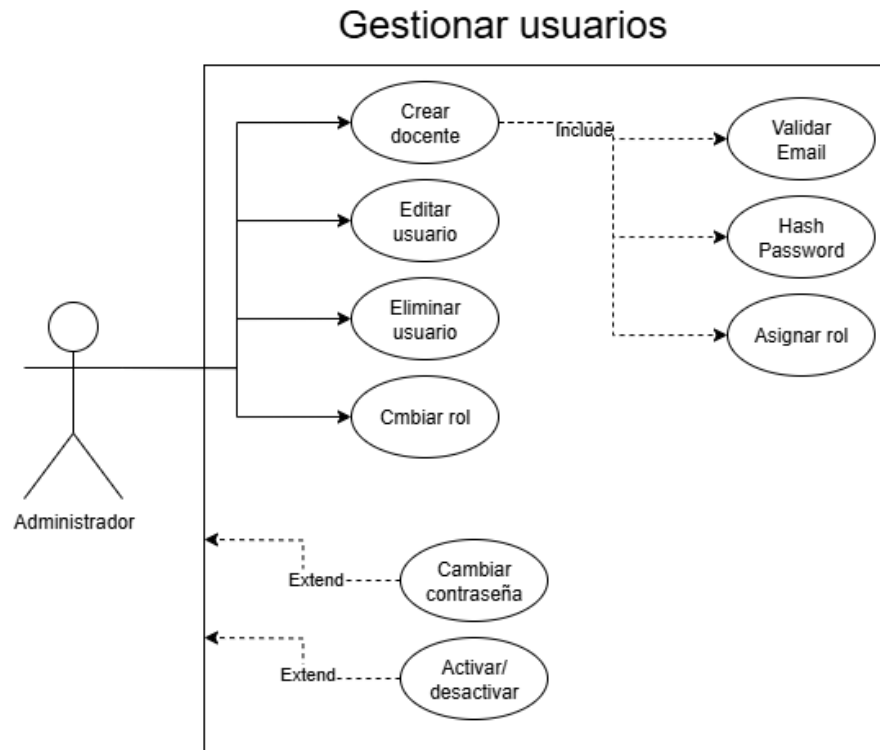
- Gestionar estudiantes



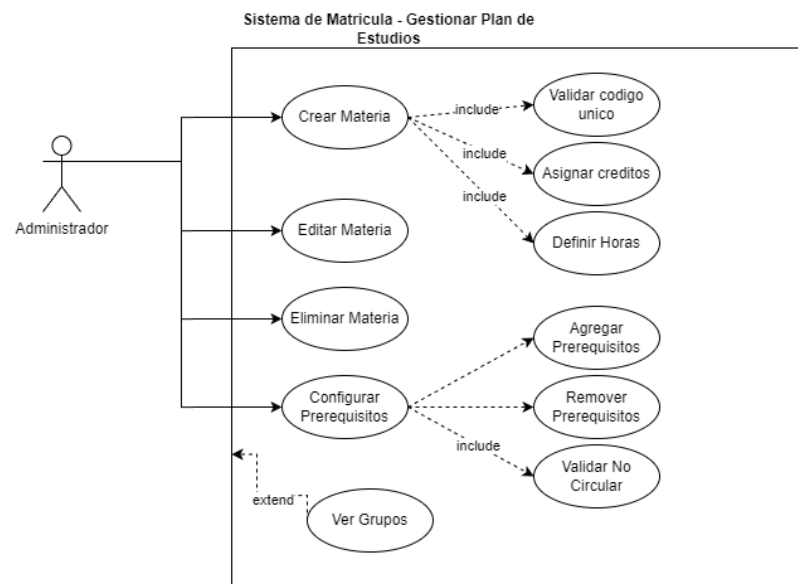
- Gestionar docentes



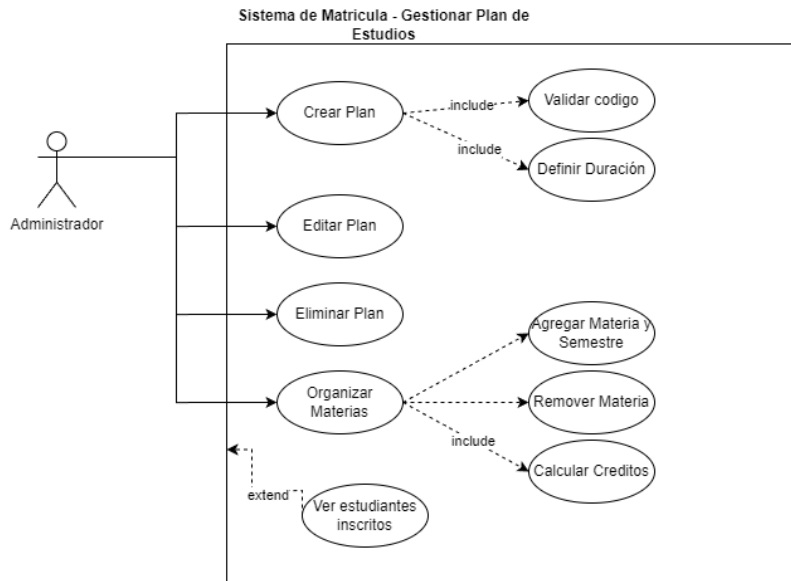
- Gestionar usuarios



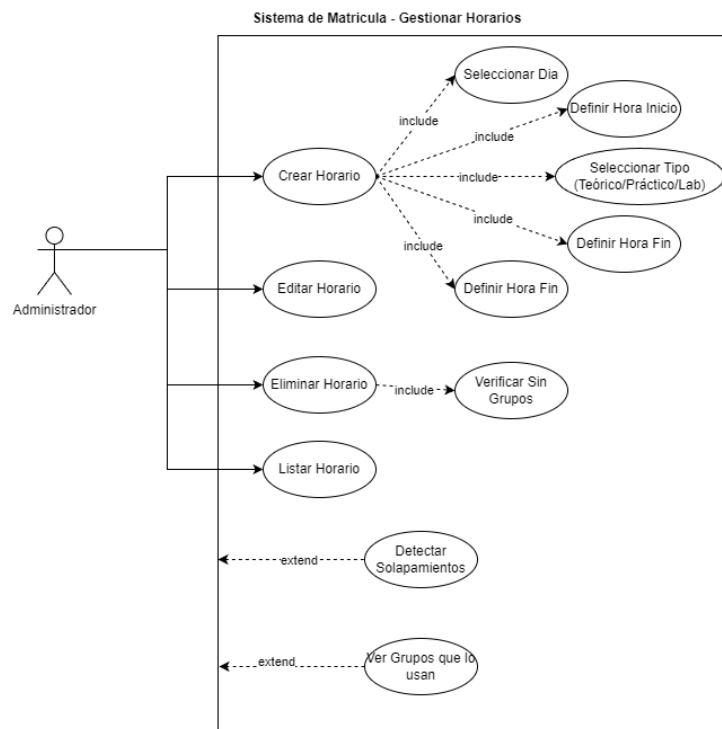
- Gestionar materias



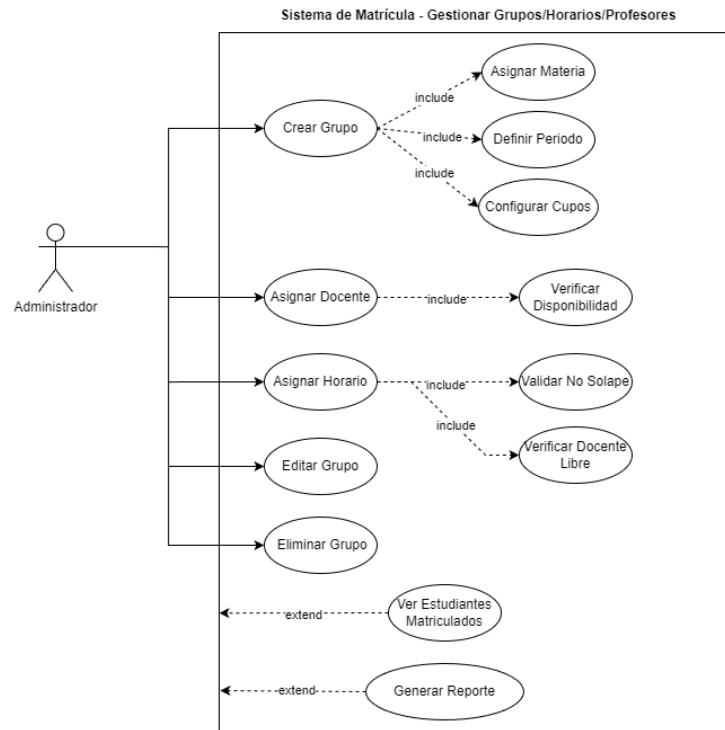
- Gestionar plan de estudios



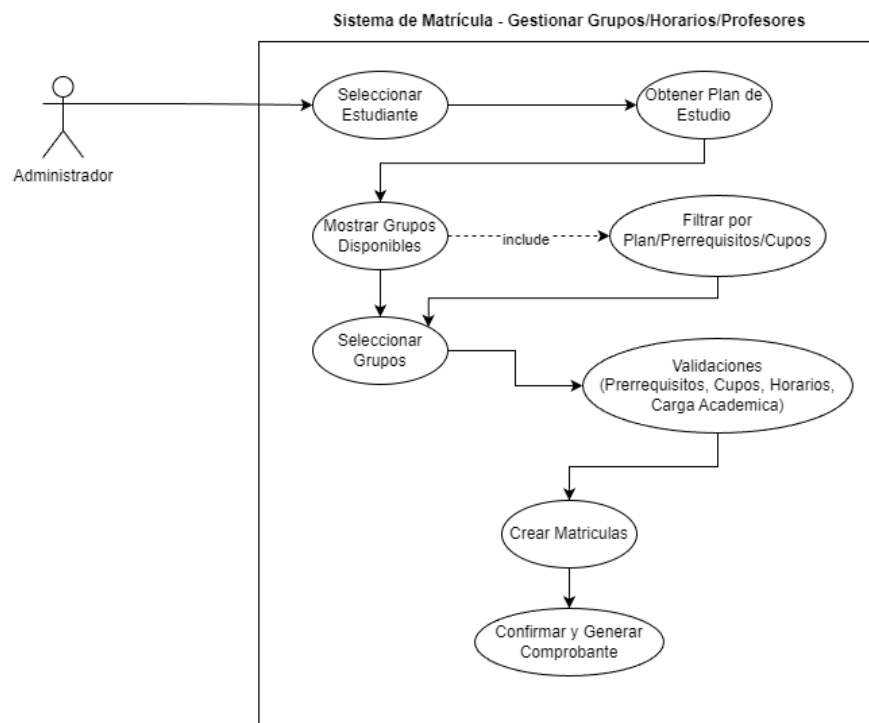
- Gestionar horarios



- Gestionar grupos-horarios-profesores



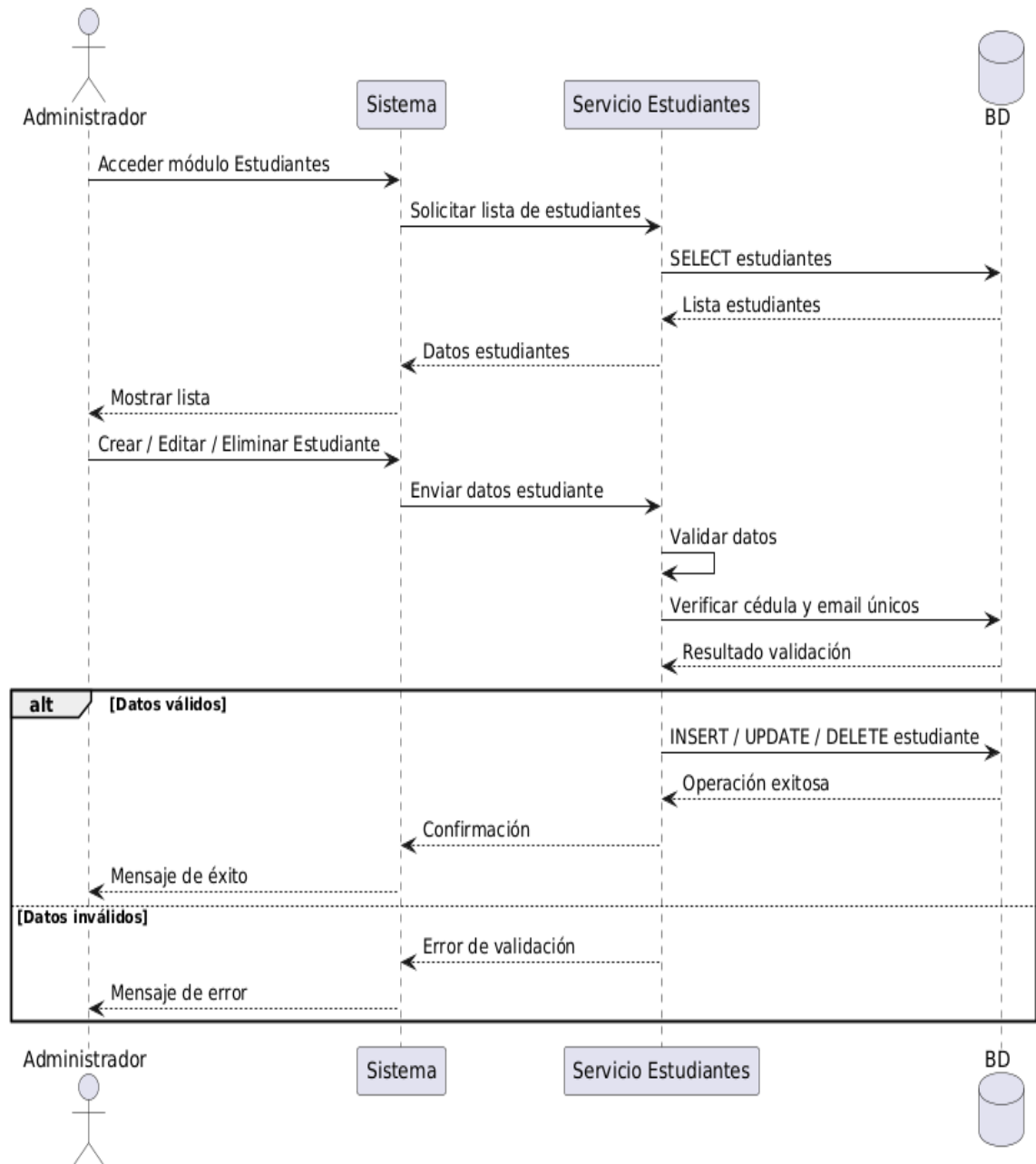
- Matricular estudiante



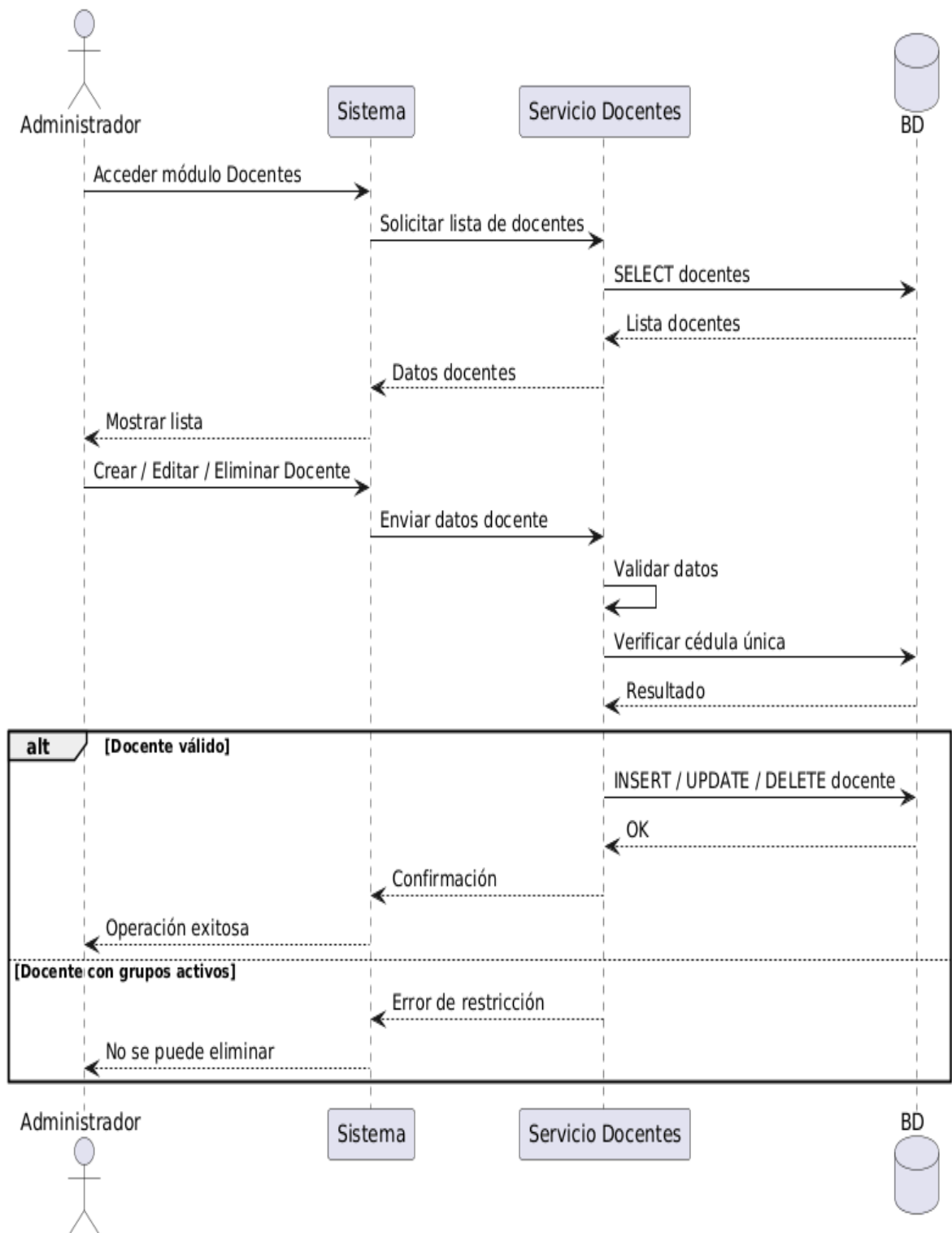
3. Diagramas de secuencia

“Estos diagramas de secuencia muestran el flujo de interacción entre los objetos del sistema a lo largo del tiempo, detallando cómo se ejecutan los procesos de gestión y matrícula desde la solicitud del usuario hasta la respuesta del sistema.”

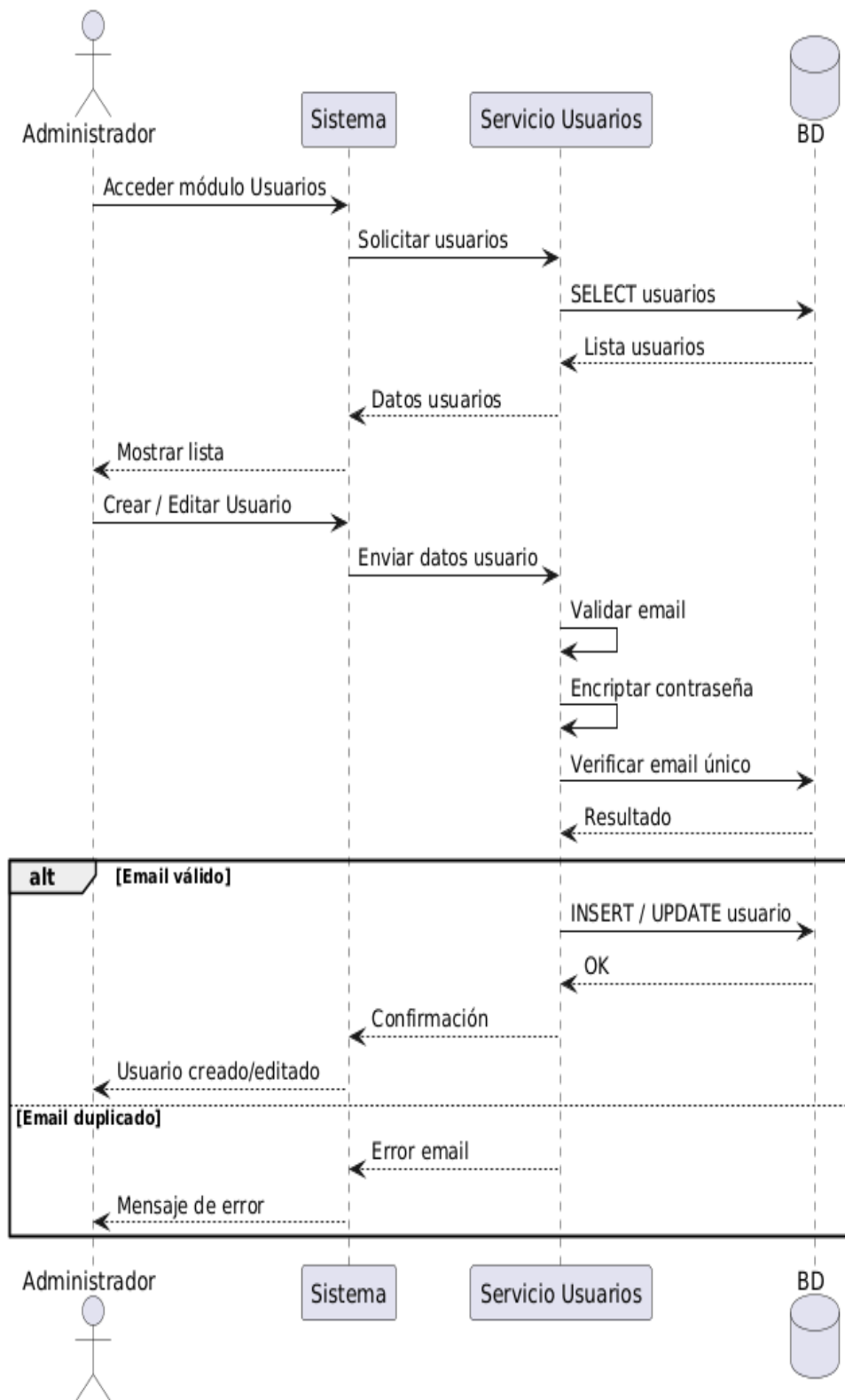
- Gestionar estudiantes



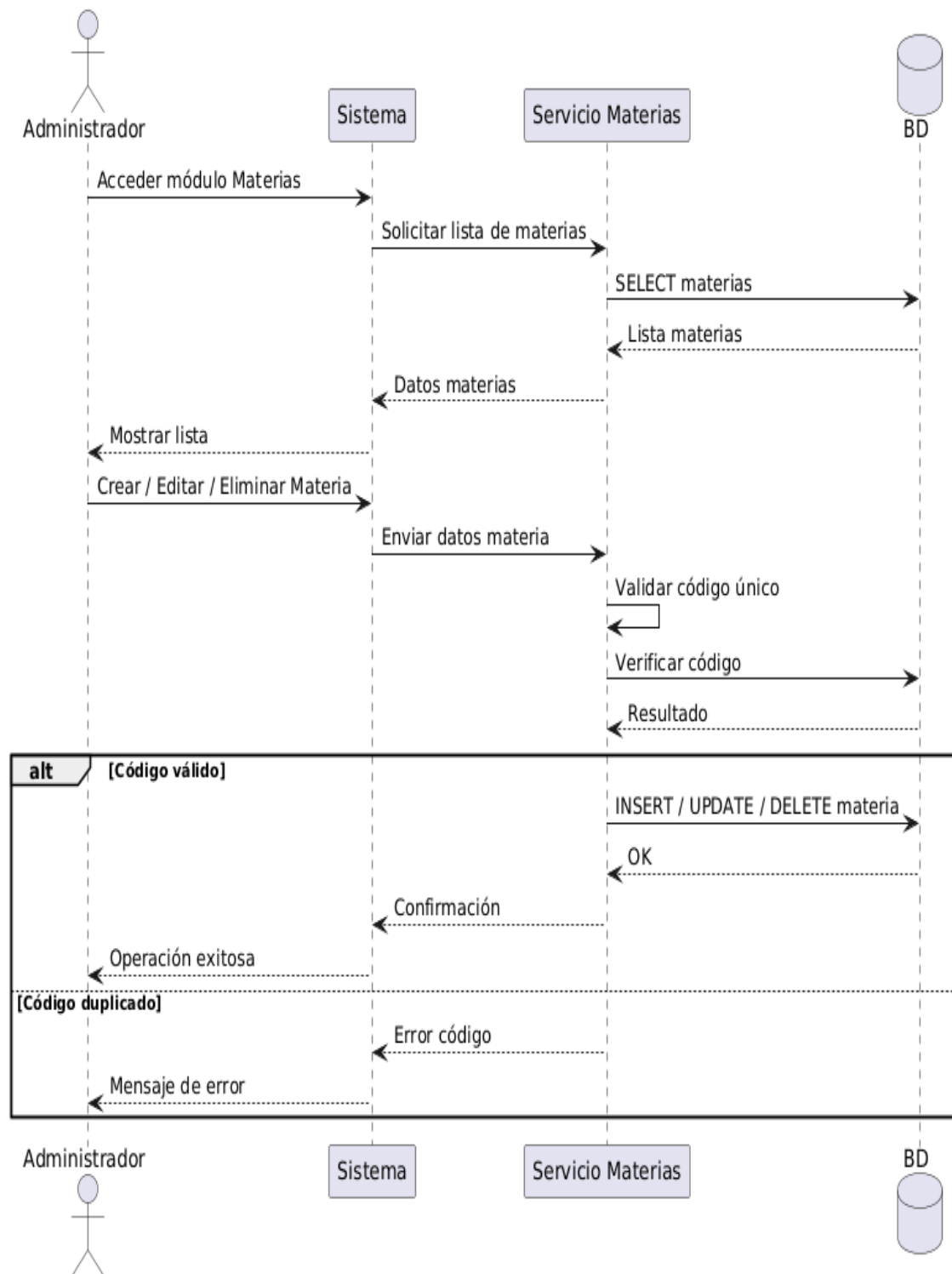
- Gestionar docentes



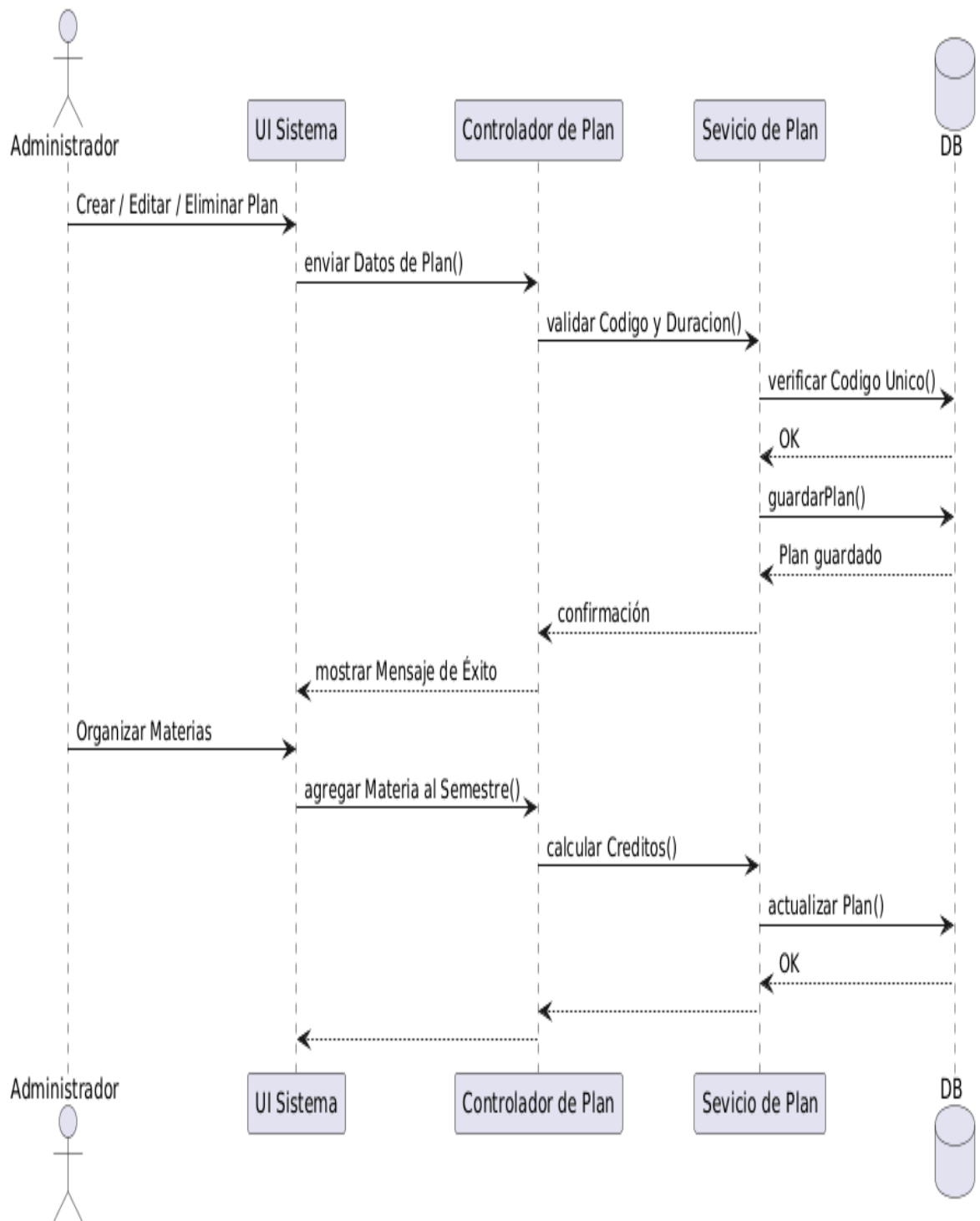
- Gestionar usuarios



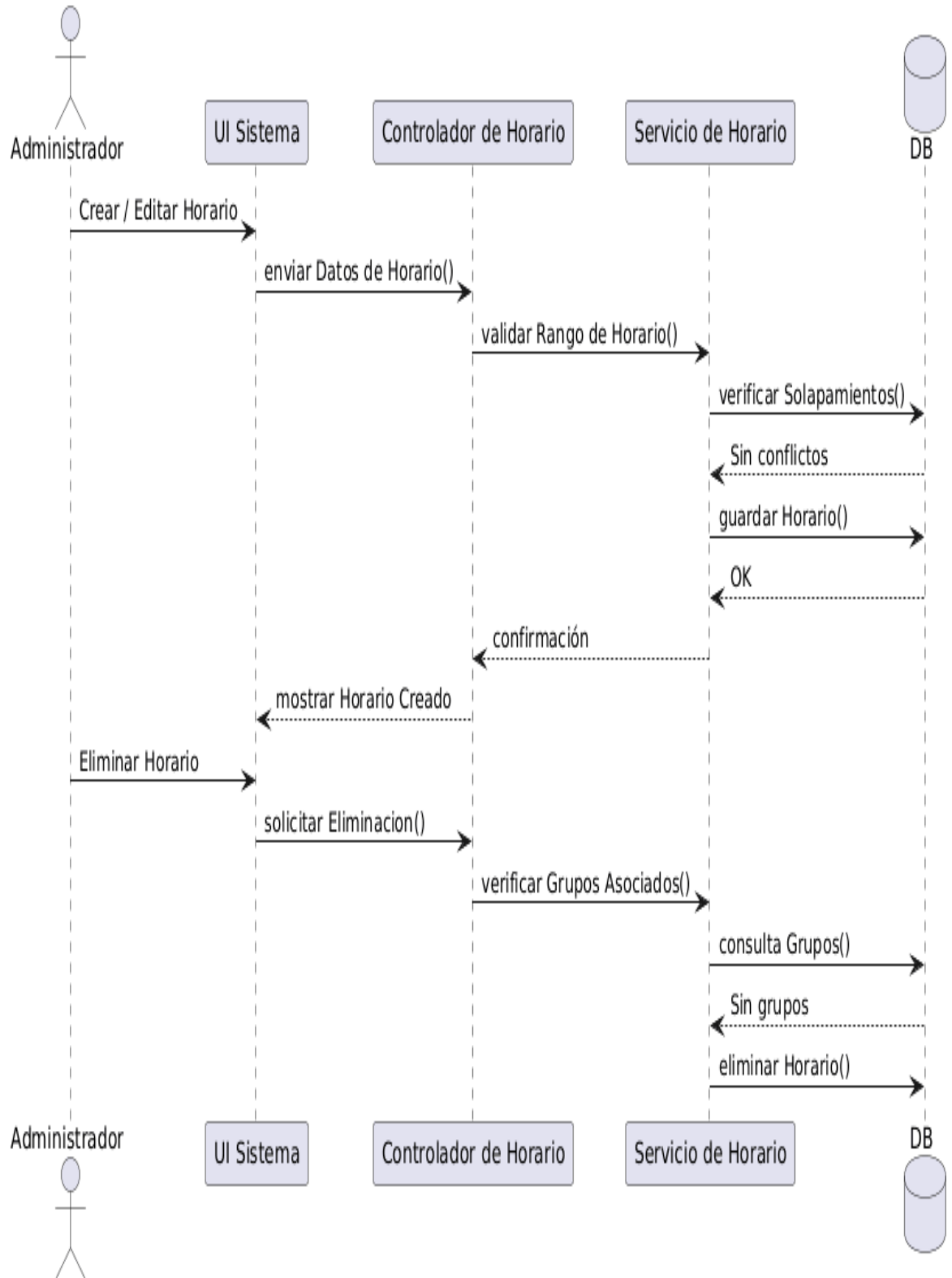
- Gestionar materias



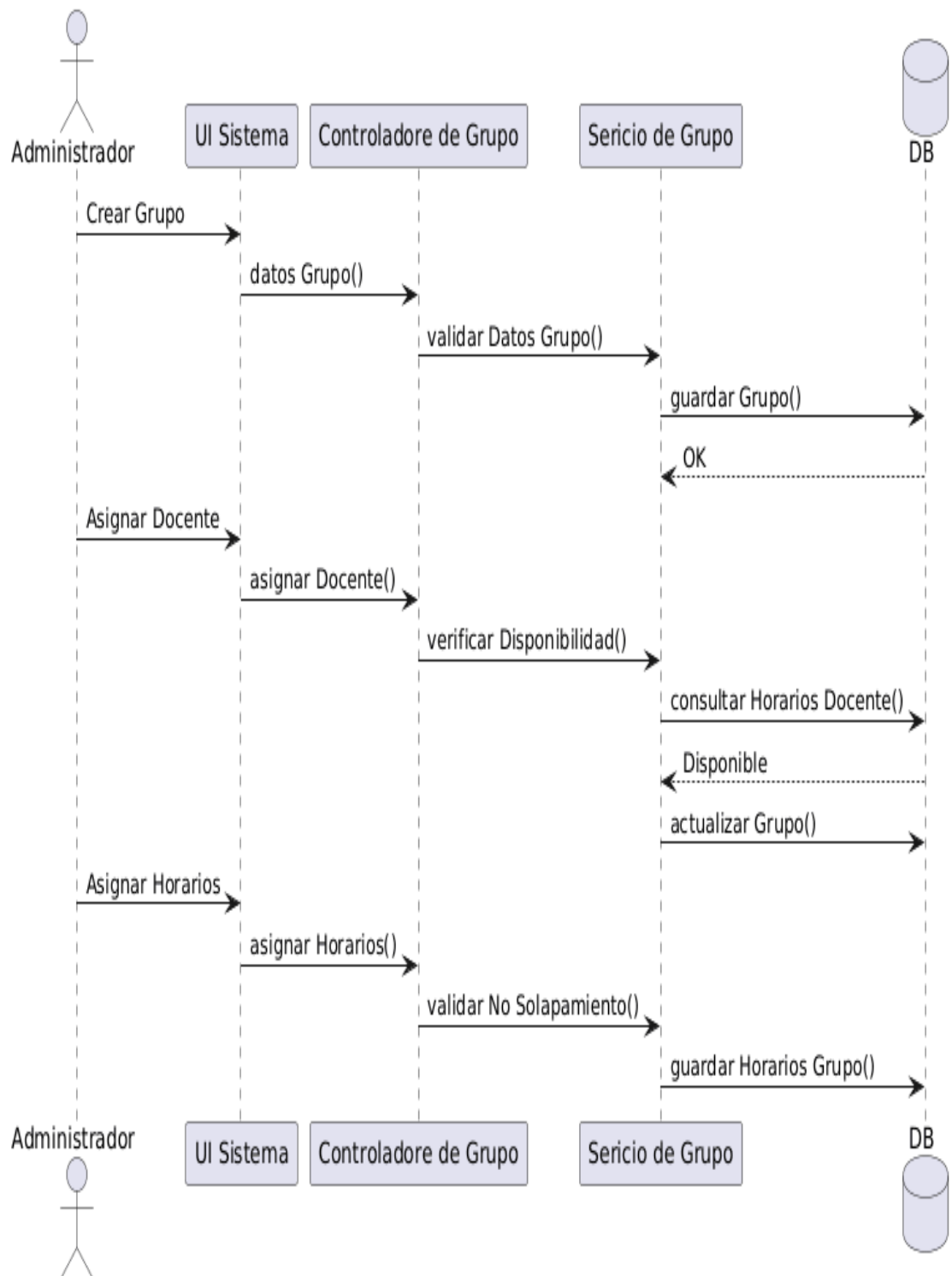
- Gestionar plan de estudios



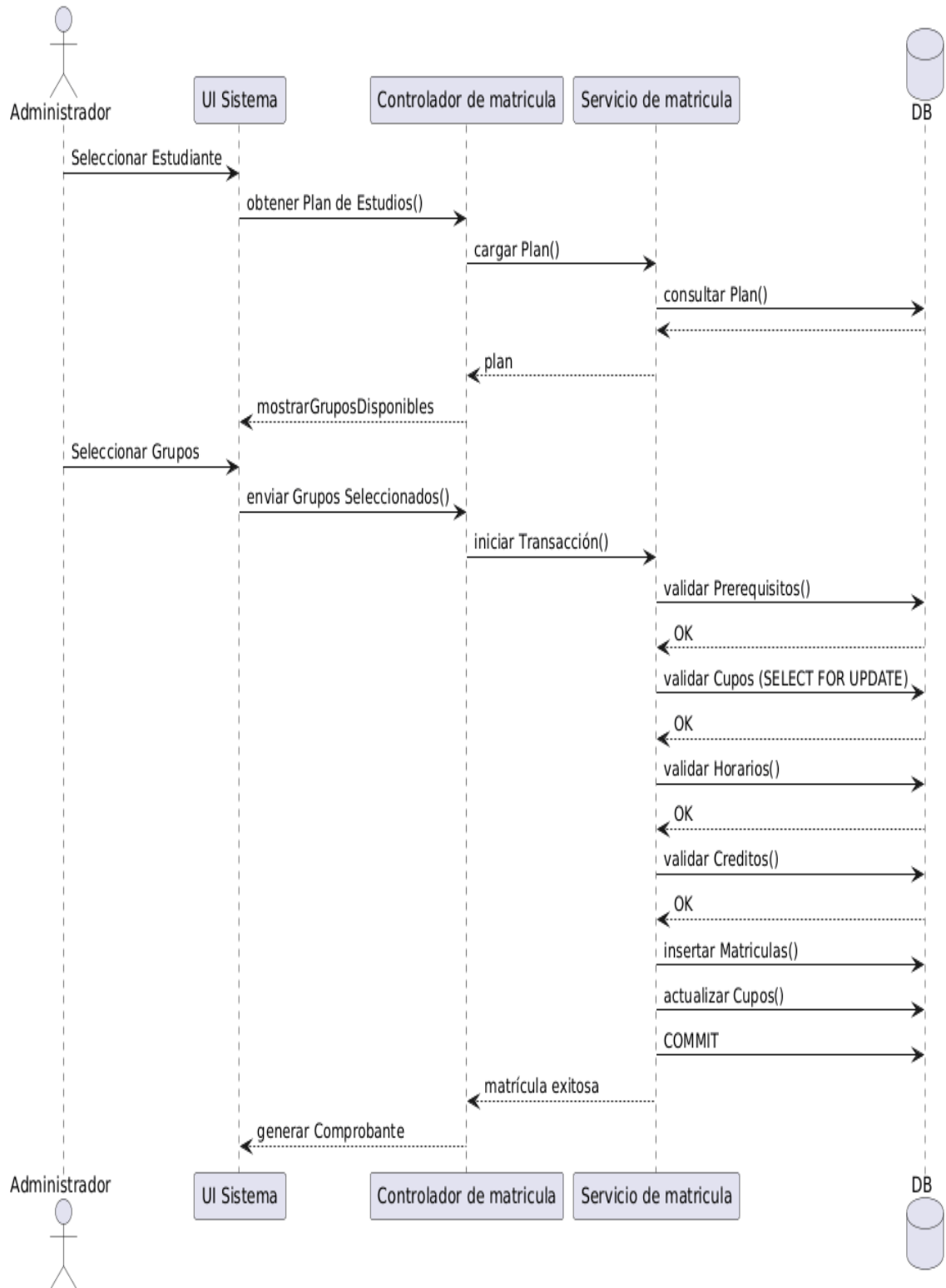
- Gestionar horarios



- Gestionar grupos-horarios-profesores



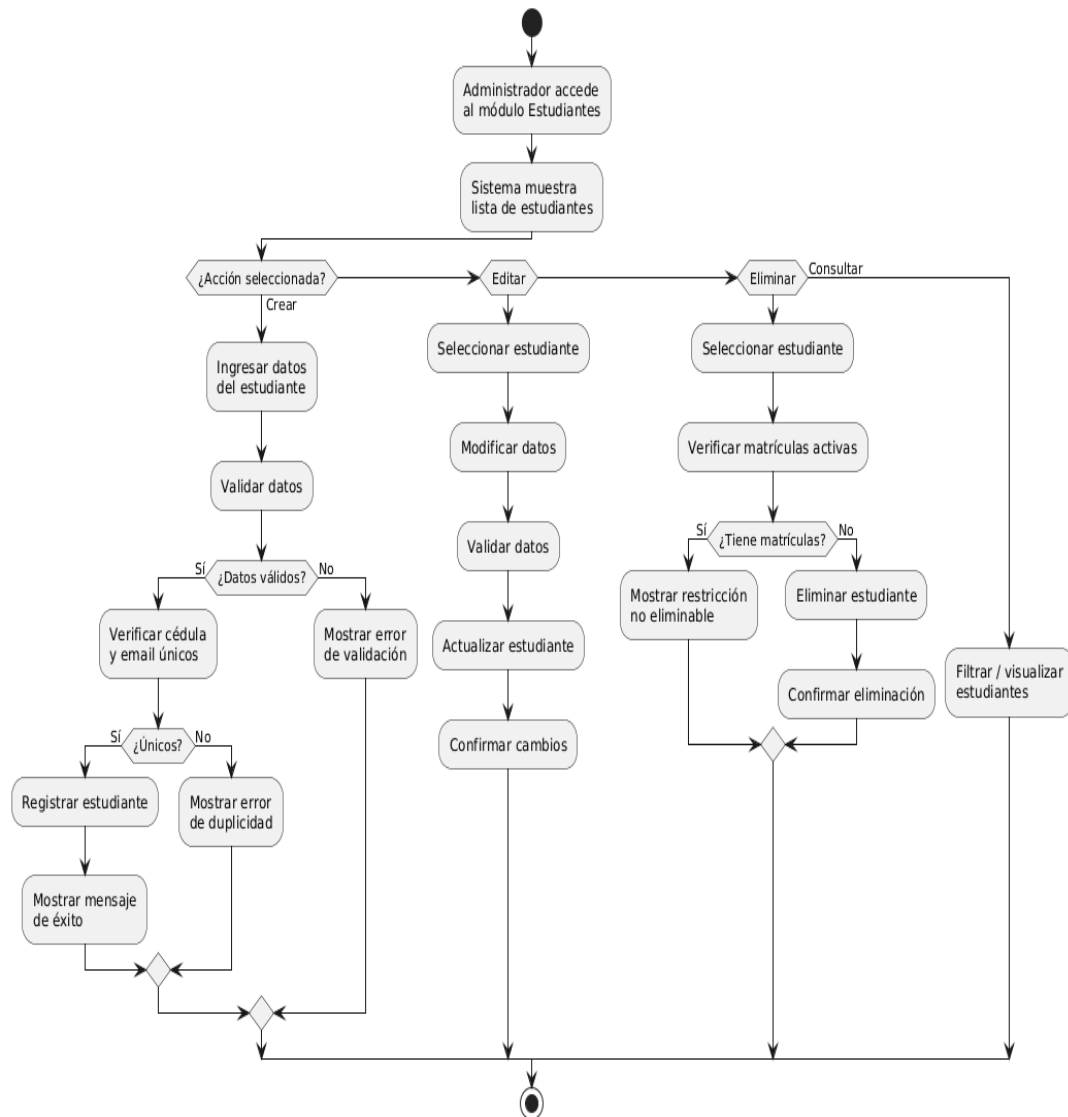
- Matricular estudiante



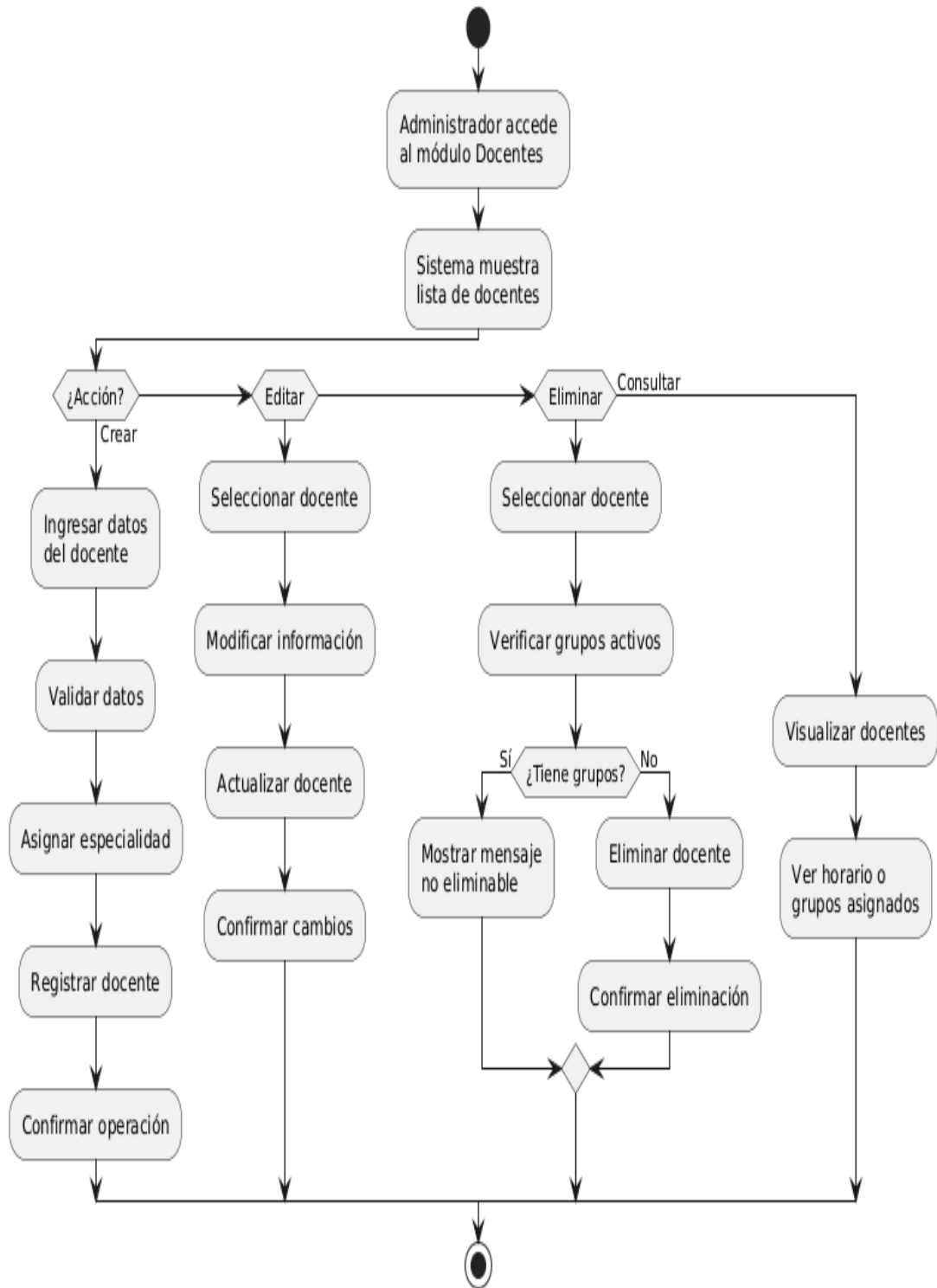
4. Diagramas de actividad

"Aqui en los diagramas de actividad describimos el flujo de trabajo interno de cada proceso, permitiendo visualizar las decisiones, validaciones y pasos necesarios para completar cada operación del sistema."

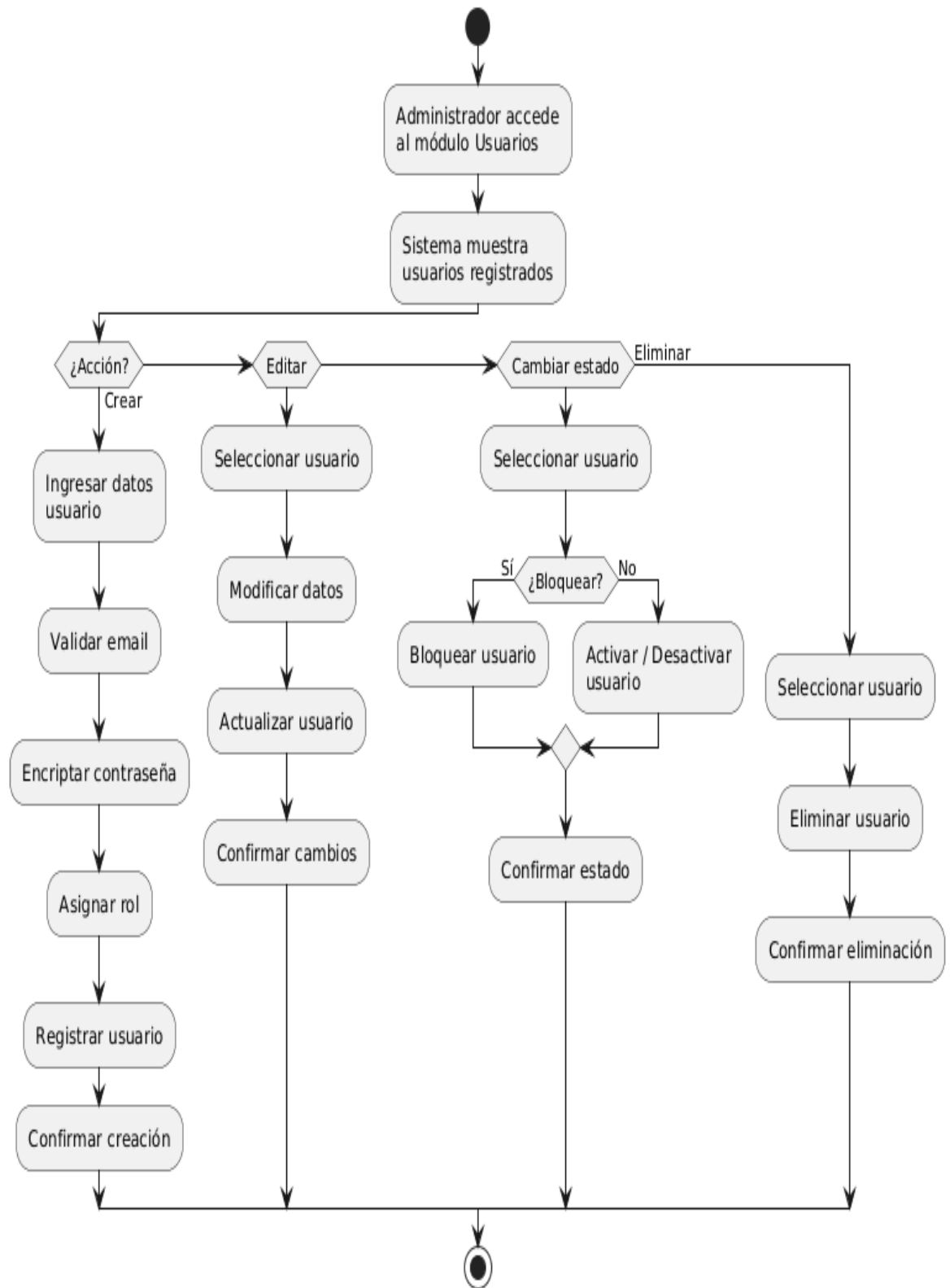
- Gestionar estudiantes



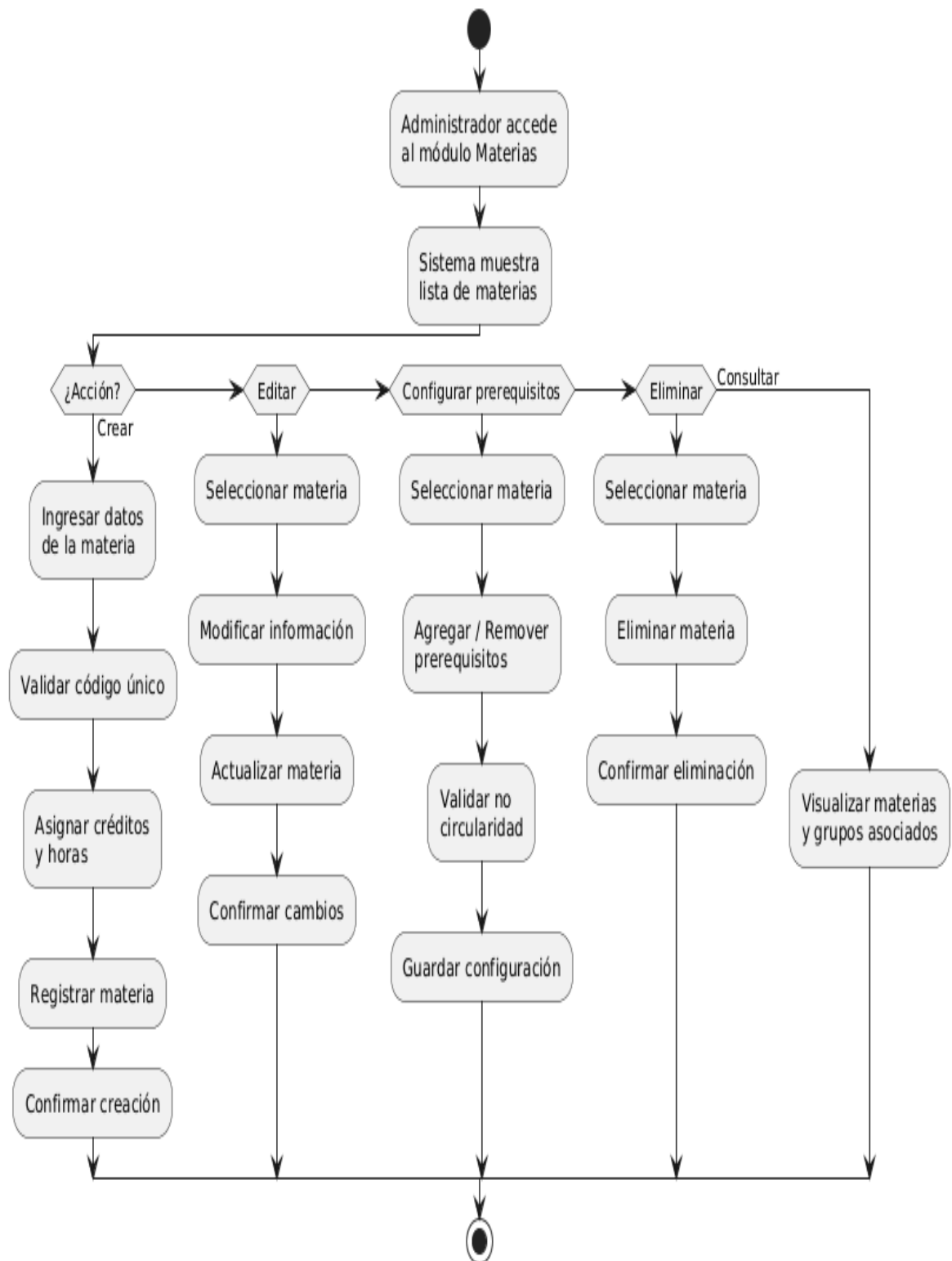
- Gestionar docentes



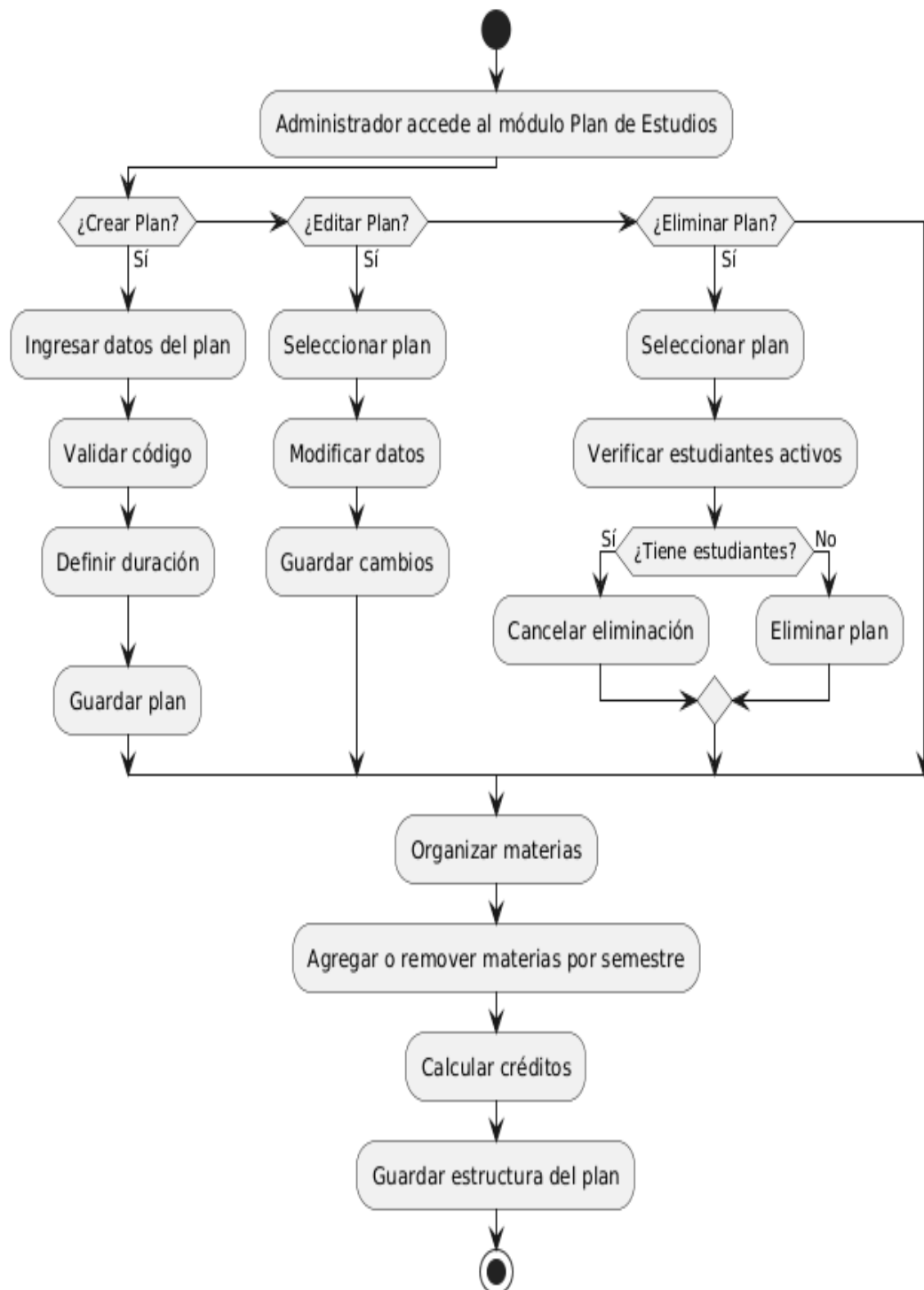
- Gestionar usuarios



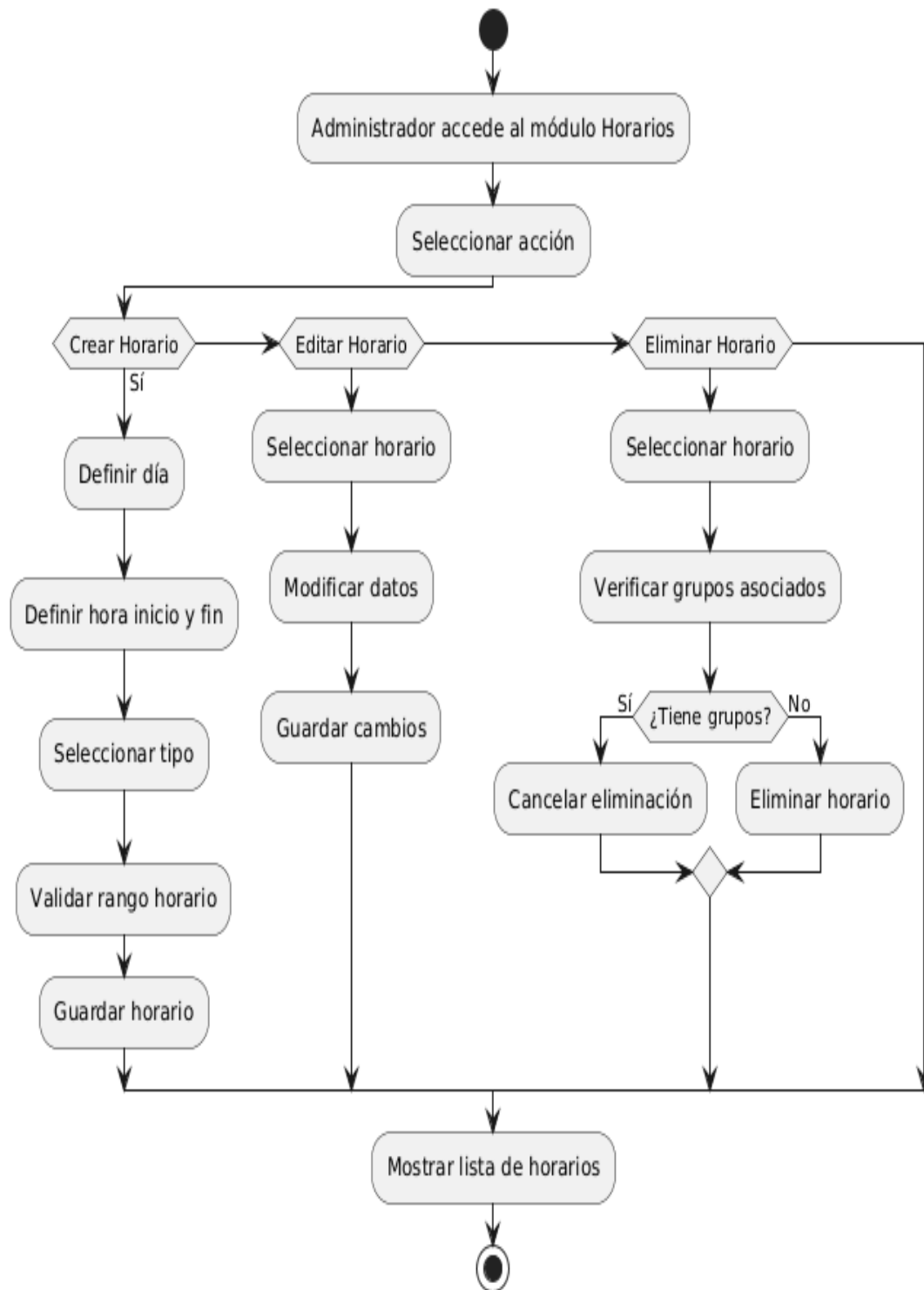
- Gestionar materias



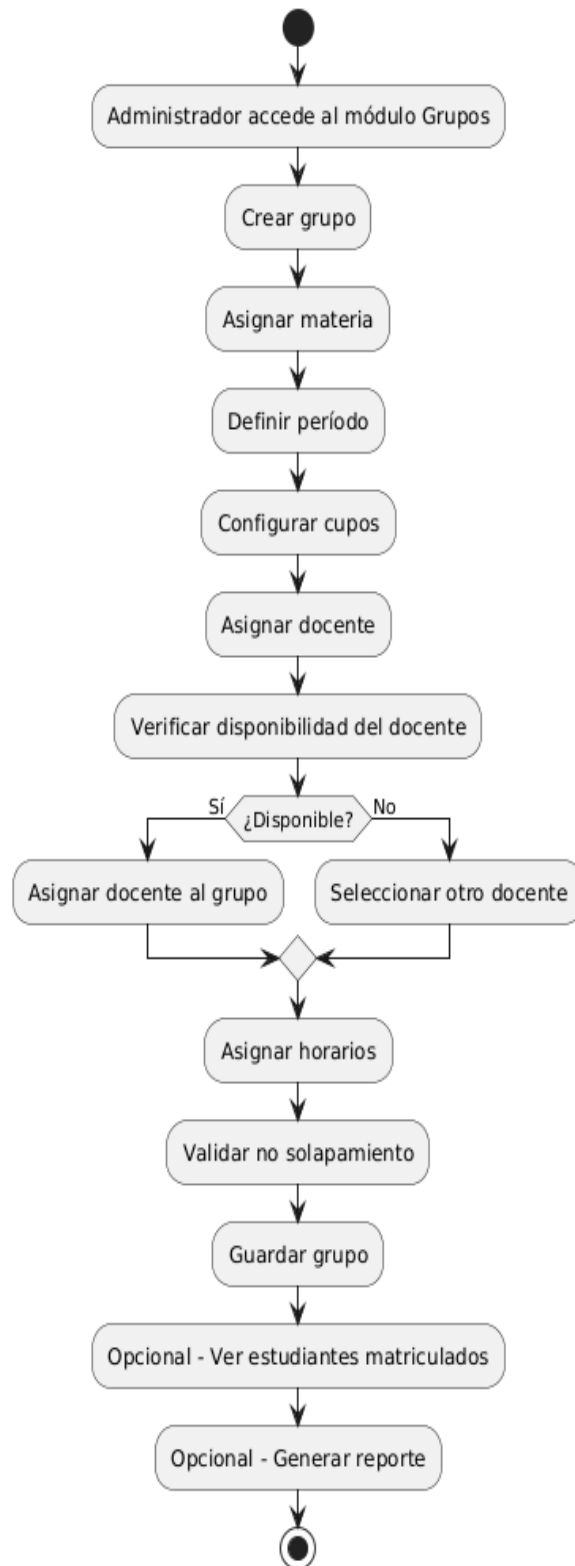
- Gestionar plan de estudios



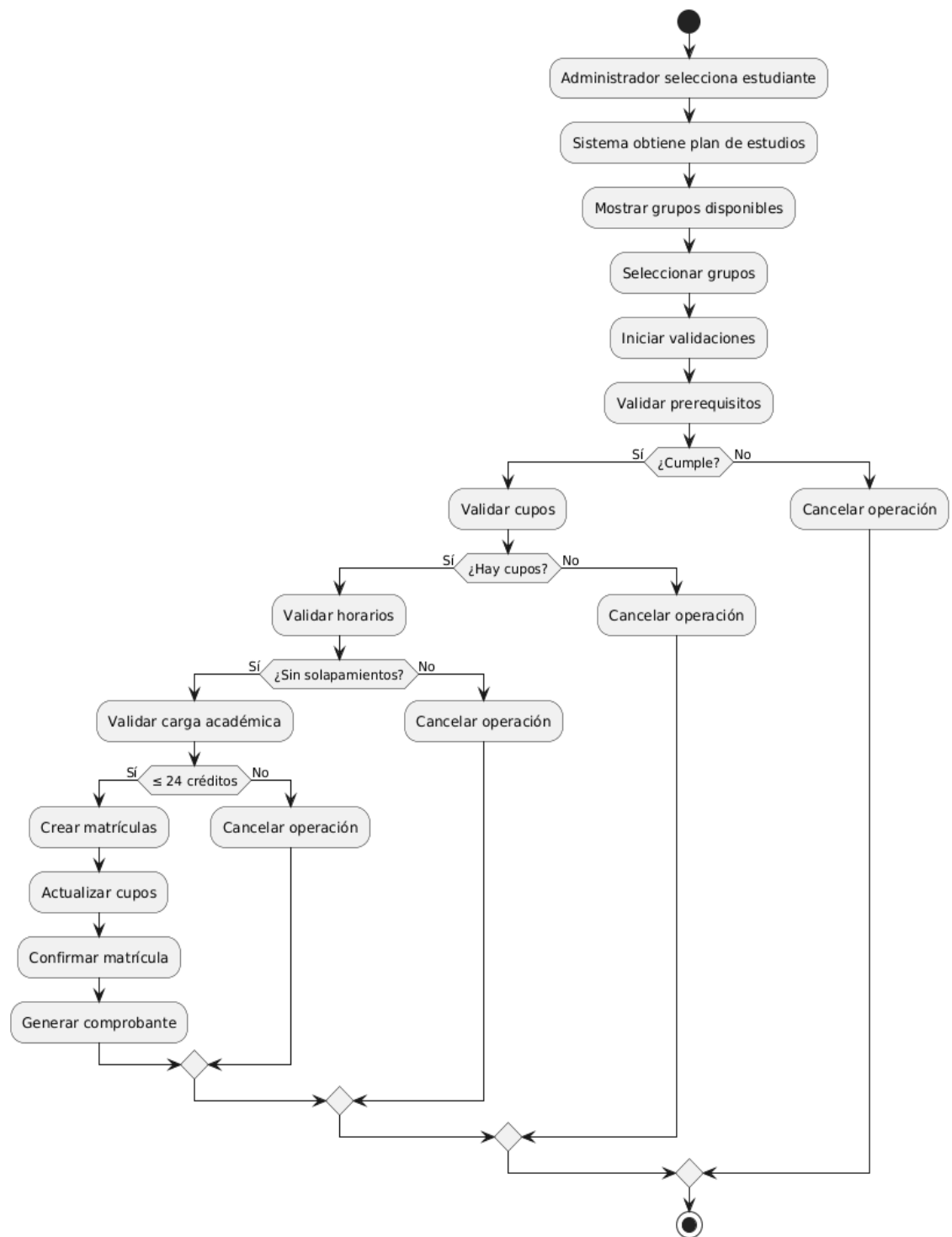
- Gestionar horarios



- Gestionar grupos-horarios-profesores



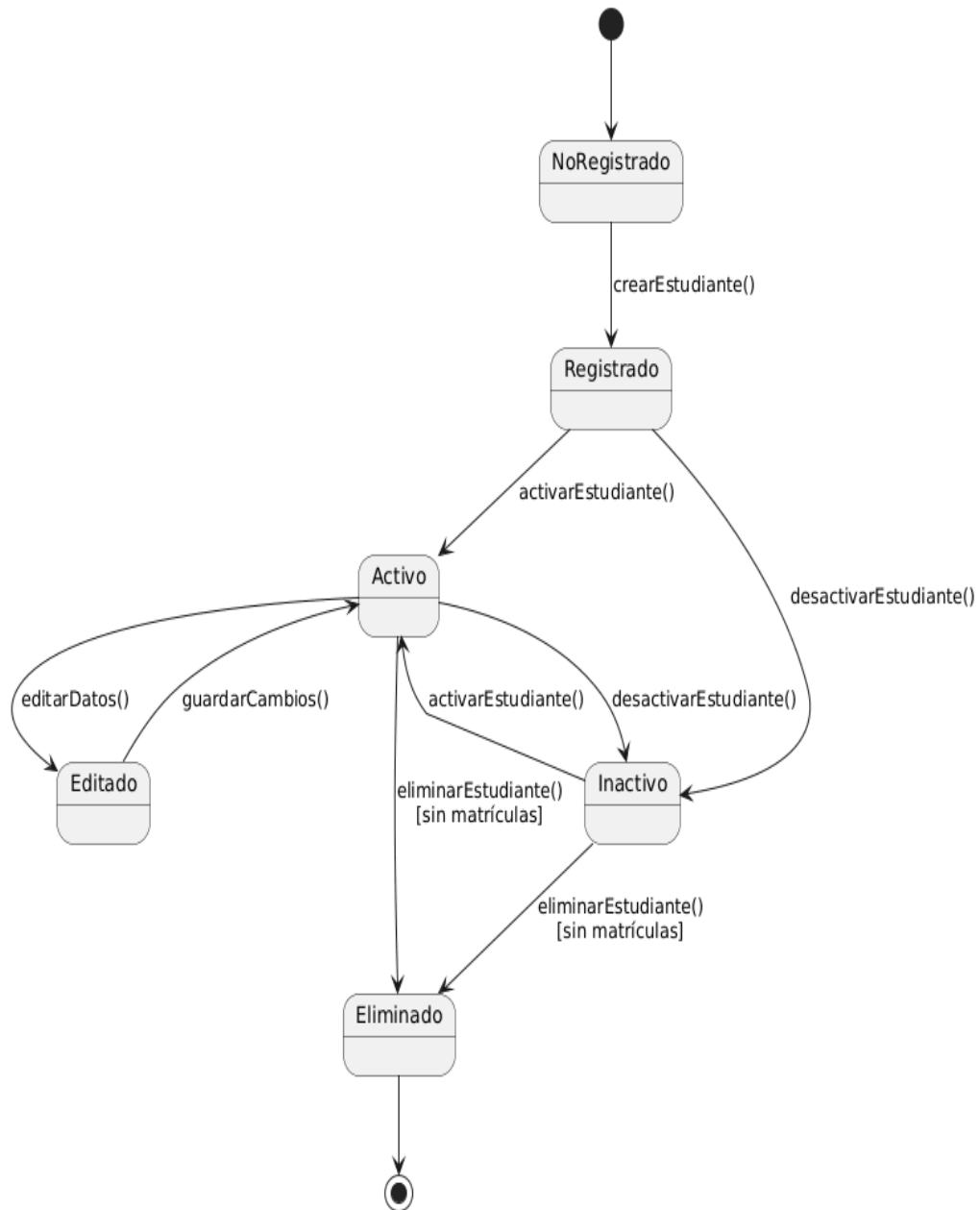
- Matricular estudiante



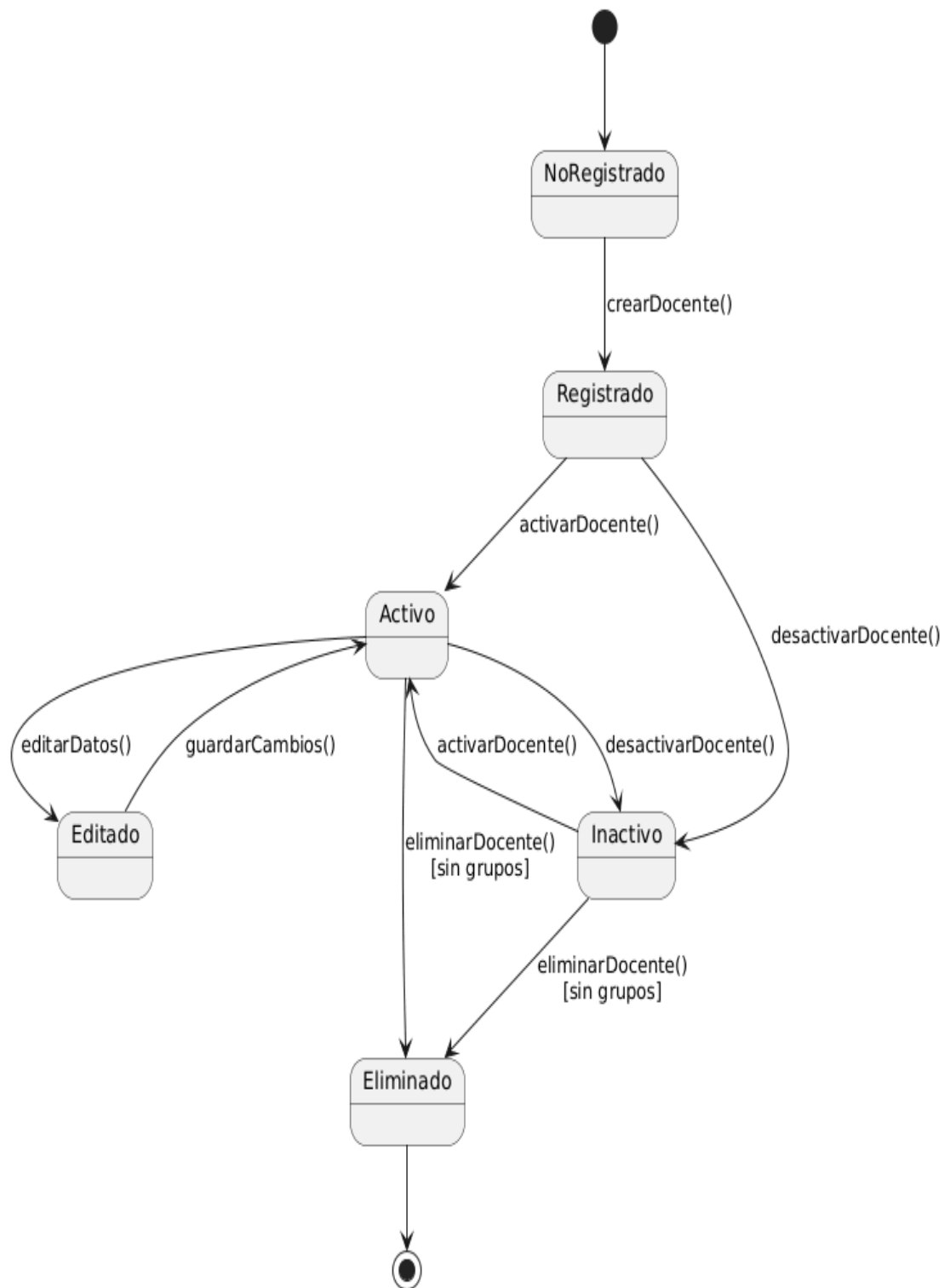
5. Diagramas de estado

"Dentro de los diagramas de estado representamos los distintos estados por los que atraviesan las entidades del sistema durante su ciclo de vida, como creación, actualización y finalización de procesos."

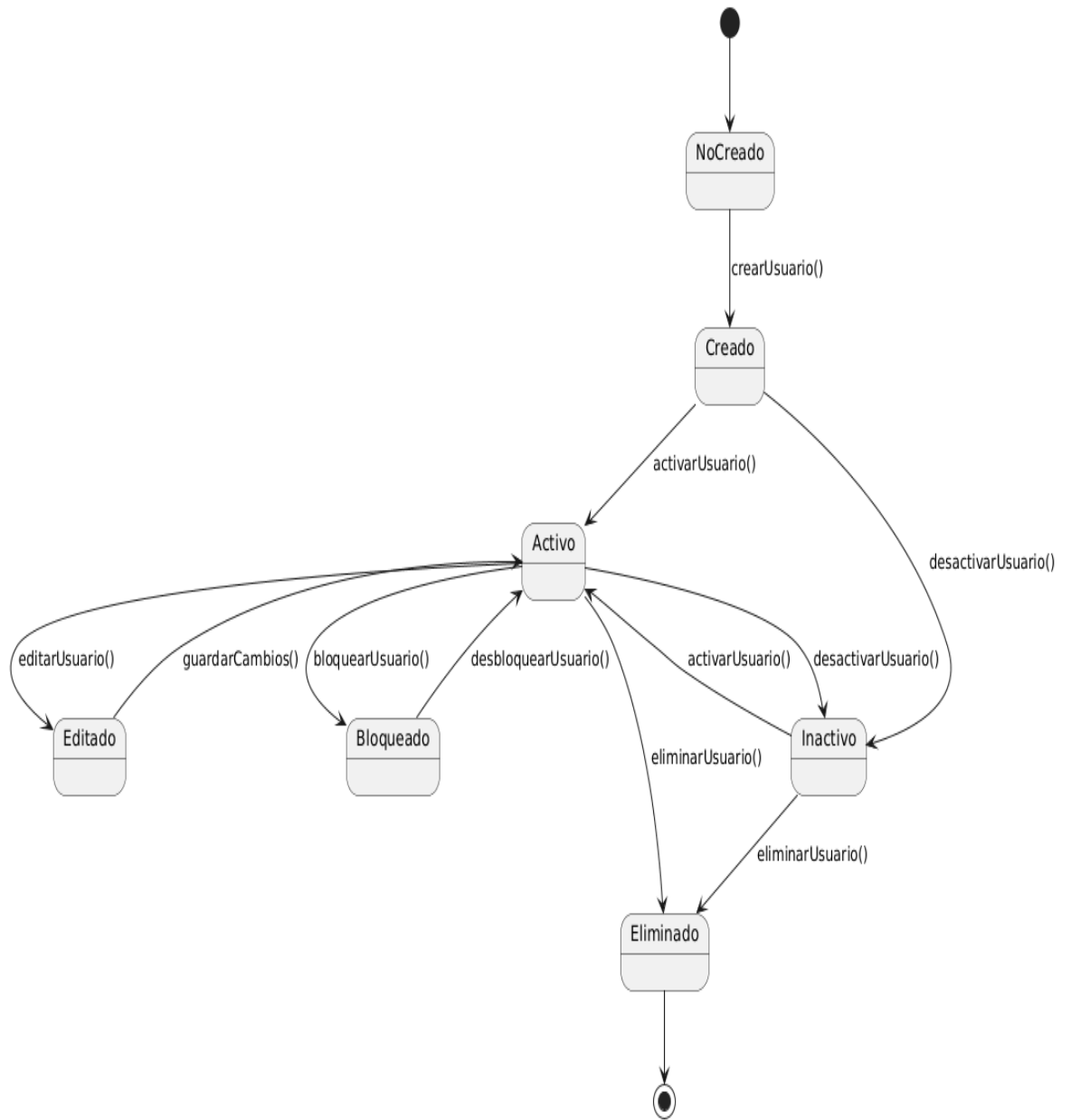
- Gestionar estudiantes



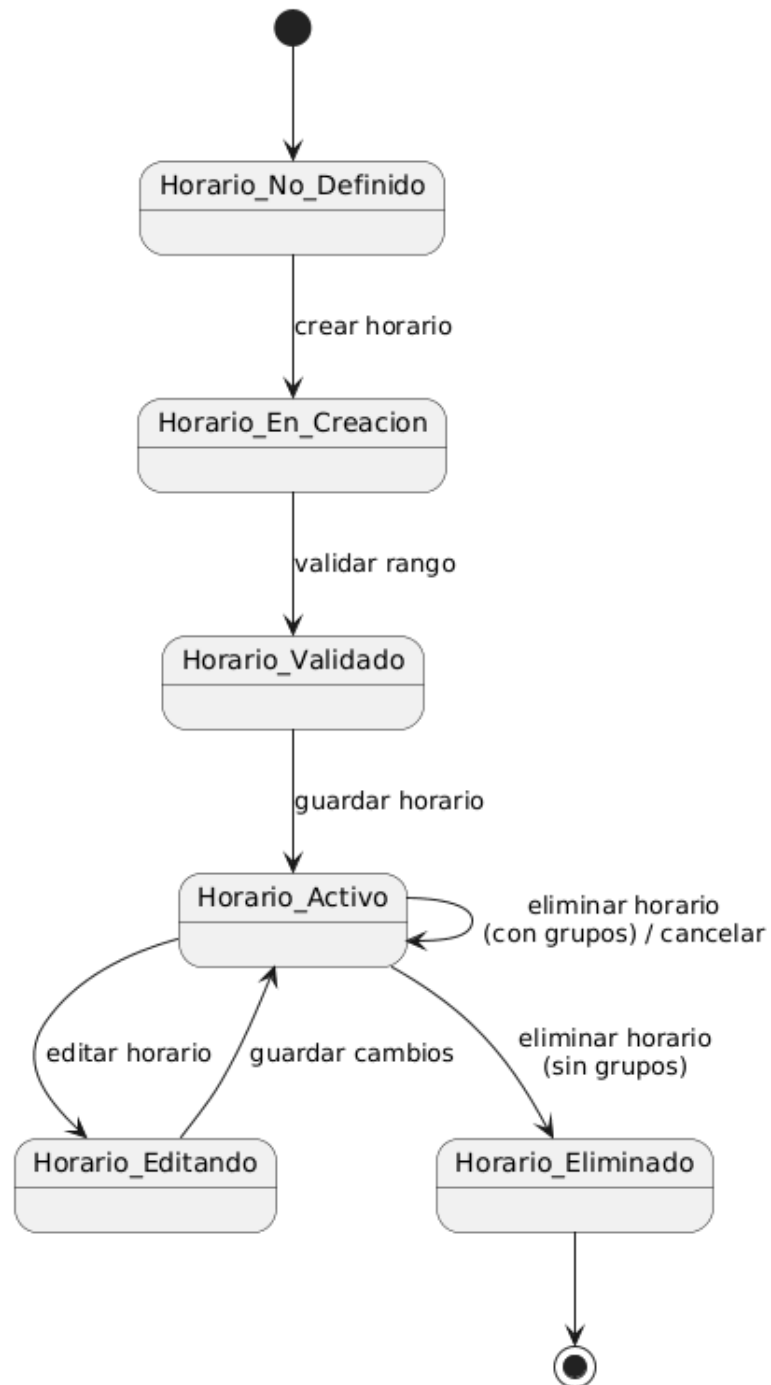
- Gestionar docentes



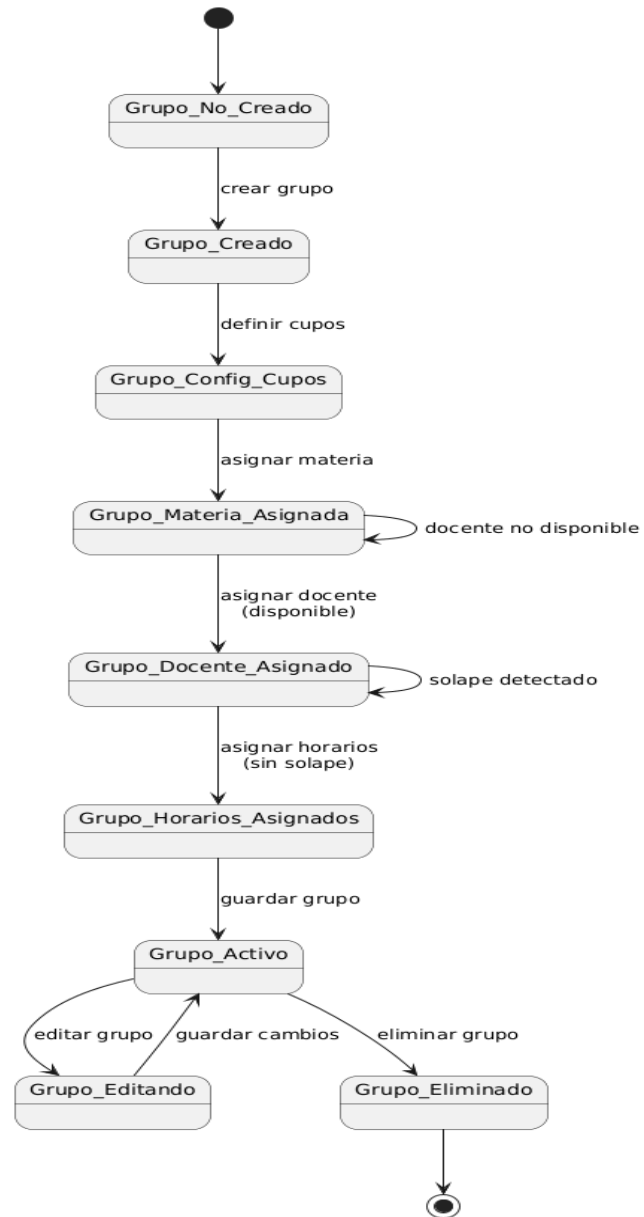
- Gestionar usuarios



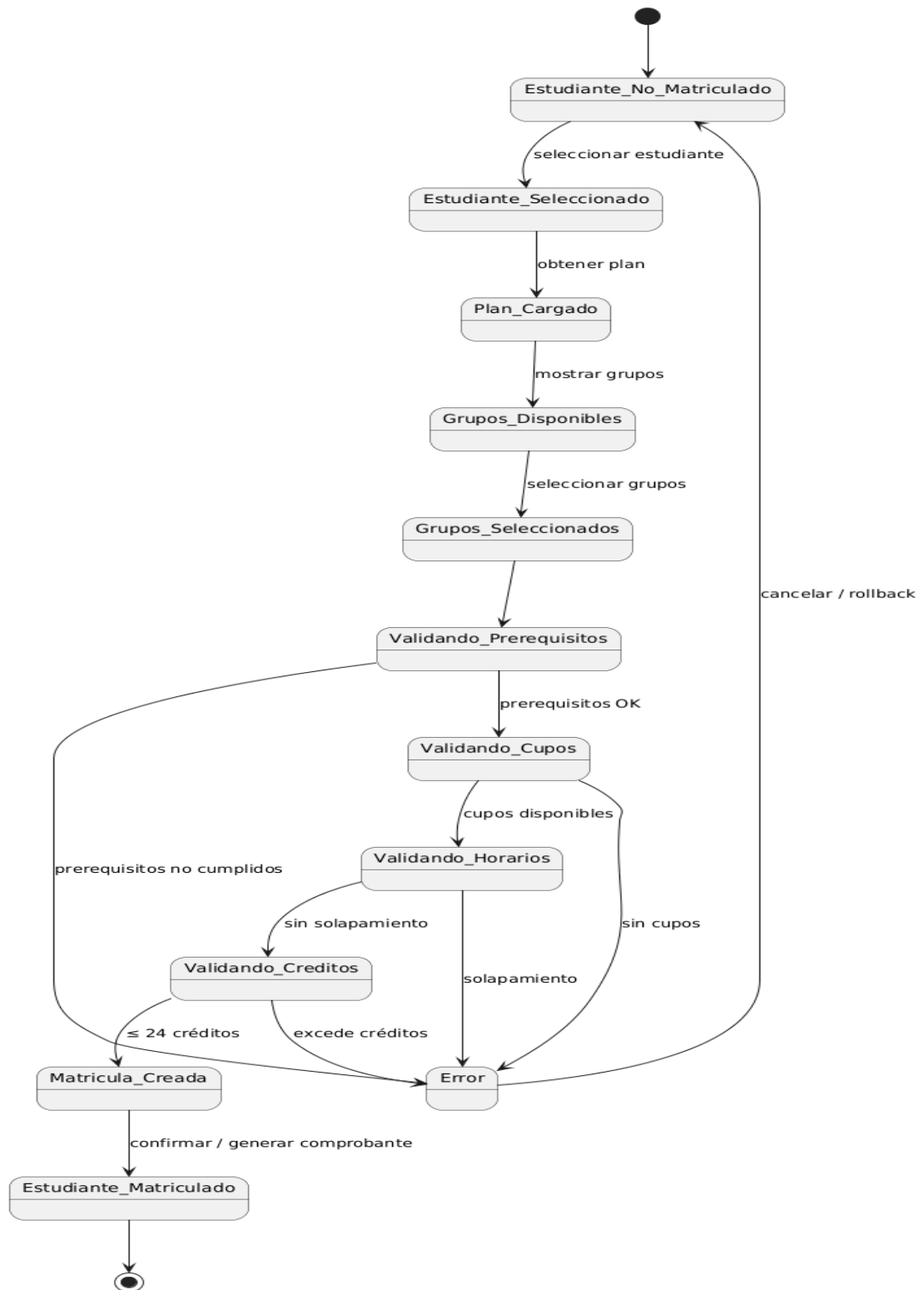
- Gestionar materias



- Gestionar grupos-horarios-profesores



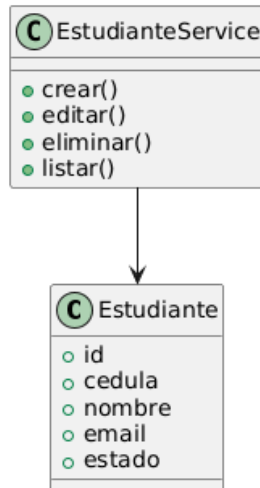
- Matricular estudiante



6. Diagramas de clase

"En los diagramas de clases definimos la estructura estática del sistema, mostrando las clases principales, sus atributos, métodos y las relaciones existentes entre ellas, sirviendo como base para la implementación del sistema."

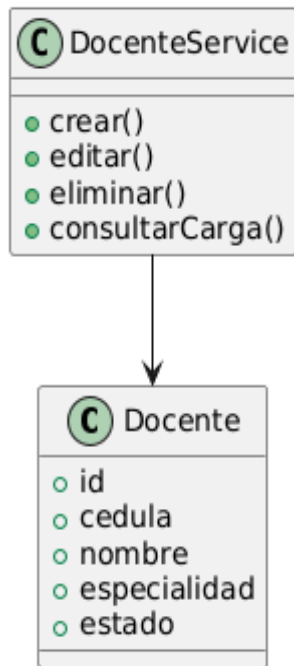
- Gestionar estudiantes



En este diagrama de clases se representa la relación entre las clases Estudiante, Usuario y Matrícula.

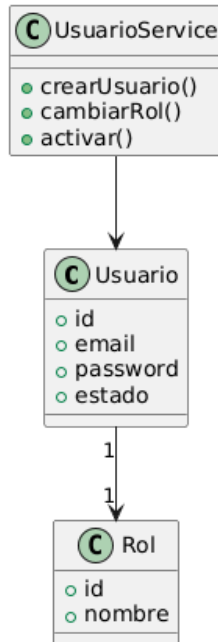
- La clase Estudiante mantiene una relación de asociación con la clase Usuario, con una cardinalidad 1 a 1, ya que cada estudiante posee exactamente una cuenta de usuario para acceder al sistema, y cada usuario estudiante está vinculado a un solo estudiante.
- Asimismo, existe una relación de asociación entre Estudiante y Matrícula con cardinalidad 1 a muchos (1..*), dado que un estudiante puede realizar múltiples matrículas a lo largo de su vida académica, mientras que cada matrícula pertenece a un único estudiante.
- La dependencia funcional indica que la clase Matrícula depende de Estudiante, ya que no puede existir una matrícula sin un estudiante previamente registrado. Esta relación refleja el control académico y la trazabilidad de las inscripciones.

- Gestionar docentes



Este diagrama muestra la clase Docente relacionada con las clases Usuario y Grupo.

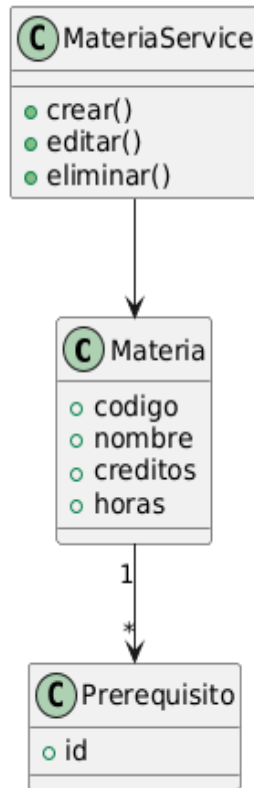
- La relación entre Docente y Usuario es una asociación uno a uno (1..1), debido a que cada docente requiere una cuenta de usuario para acceder al sistema.
 - Entre Docente y Grupo existe una relación de asociación uno a muchos (1..*), ya que un docente puede estar asignado a varios grupos académicos, mientras que cada grupo solo puede tener un docente asignado.
 - La dirección de la relación indica que el grupo depende del docente para su funcionamiento académico, reflejando la asignación de carga docente y la planificación de horarios.
-
- Gestionar usuarios



En este diagrama se define la clase Usuario, la cual mantiene una relación de dependencia con la clase Rol.

- La cardinalidad es muchos a uno (*..1), ya que múltiples usuarios pueden compartir un mismo rol (Administrador, Docente o Estudiante), pero cada usuario solo puede tener un rol activo a la vez.
- La relación es de dependencia porque los permisos y accesos del usuario están condicionados por el rol asignado. Sin un rol definido, el usuario no puede operar dentro del sistema, garantizando la seguridad y el control de accesos.

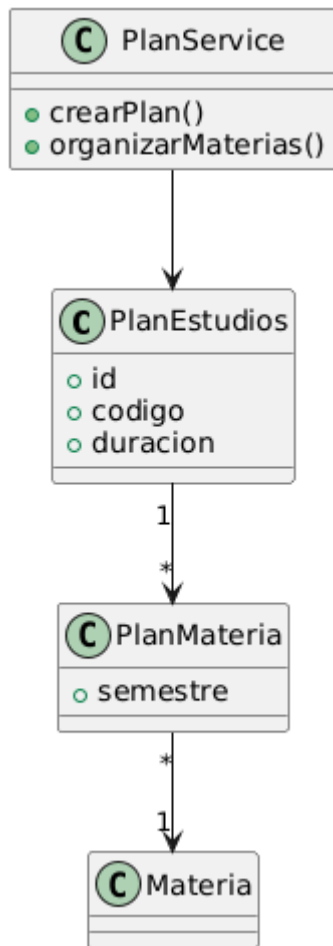
- Gestionar materias



Este diagrama presenta la clase Materia con una relación de agregación consigo misma para representar los prerrequisitos.

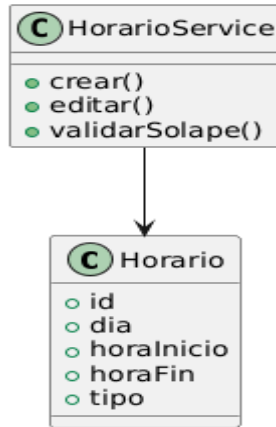
- La cardinalidad es muchos a muchos (..), ya que una materia puede tener varios prerrequisitos y, a su vez, una materia puede ser prerrequisito de otras.
- La agregación indica que las materias prerrequisito existen independientemente de la materia principal, pero forman parte de su estructura académica. Esta relación permite aplicar validaciones académicas antes de permitir la matrícula del estudiante.

- Gestionar plan de estudios



En este diagrama se observa una relación de agregación entre la clase PlanEstudio y la clase Materia, con cardinalidad 1 a muchos (1..).*

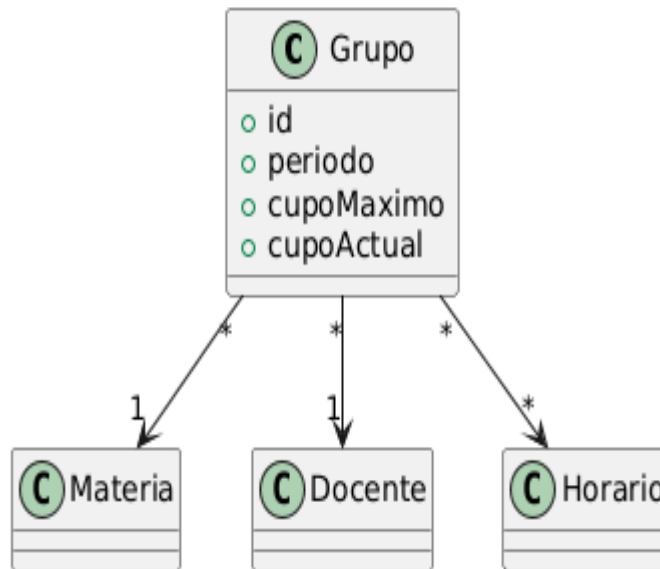
- Un plan de estudios está compuesto por varias materias, mientras que cada materia pertenece a un plan específico.
- La agregación indica que las materias pueden existir sin el plan, pero el plan de estudios las organiza académicamente. Esta relación permite estructurar la malla curricular por semestres y controlar los créditos académicos.
- Gestionar horarios



Este diagrama muestra la relación entre las clases Horario y Grupo, mediante una asociación muchos a muchos (..).

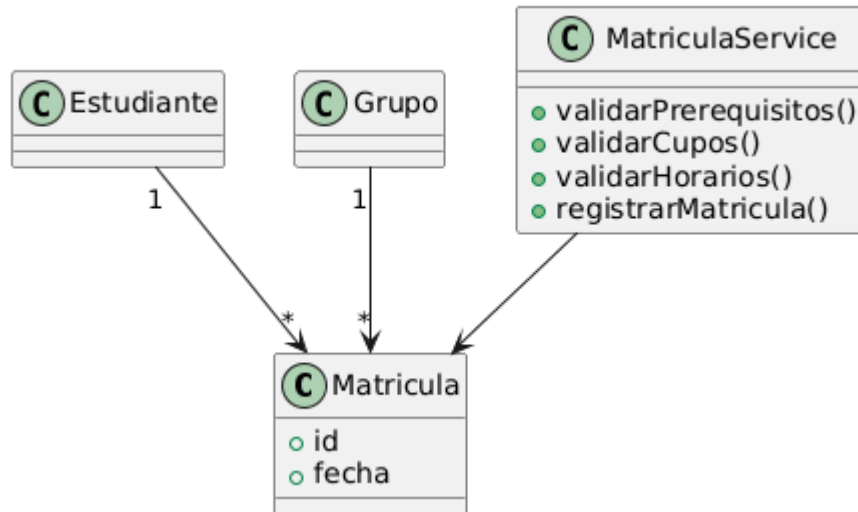
- Un grupo puede tener varios horarios asignados, y un horario puede ser utilizado por diferentes grupos en distintos períodos académicos.
- La relación refleja la reutilización de bloques horarios y permite detectar solapamientos de tiempo, lo cual es fundamental para evitar conflictos académicos durante la asignación de grupos.

- Gestionar grupos-horarios-profesores



Aquí se presenta la clase Grupo como entidad central, con relaciones hacia Materia, Docente y Horario.

- La relación entre Grupo y Materia es una asociación uno a uno (1..1), ya que cada grupo corresponde a una materia específica.
 - Con Docente, la relación es uno a muchos (1..*), permitiendo que un docente tenga múltiples grupos, pero cada grupo solo un docente.
 - Con Horario, la relación es muchos a muchos (..), permitiendo flexibilidad en la asignación de horarios.
 - Estas relaciones reflejan la estructura académica real y permiten controlar cupos, horarios y carga docente.
-
- Matricular estudiante



Este diagrama representa el proceso central del sistema, donde la clase Matrícula mantiene relaciones con Estudiante, Grupo y Materia.

- Estudiante – Matrícula: asociación 1 a muchos (1..*)
- Matrícula – Grupo: asociación muchos a uno (*..1)
- La clase Matrícula depende de Grupo para validar cupos y horarios, y de Estudiante para verificar prerrequisitos y carga académica.
- Esta dependencia refleja una relación transaccional crítica, donde todas las validaciones deben cumplirse para que la matrícula sea confirmada; de lo contrario, la operación se revierte.
- Este diseño garantiza la integridad académica y la coherencia de los datos dentro del sistema.

7. Diagramas de componente

" Cada diagrama de componentes corresponde a un caso de uso específico, permitiendo una trazabilidad directa entre los requisitos funcionales y los componentes de software que los implementan. Esta separación facilita la comprensión del sistema, el mantenimiento del código y la posible reutilización de componentes "

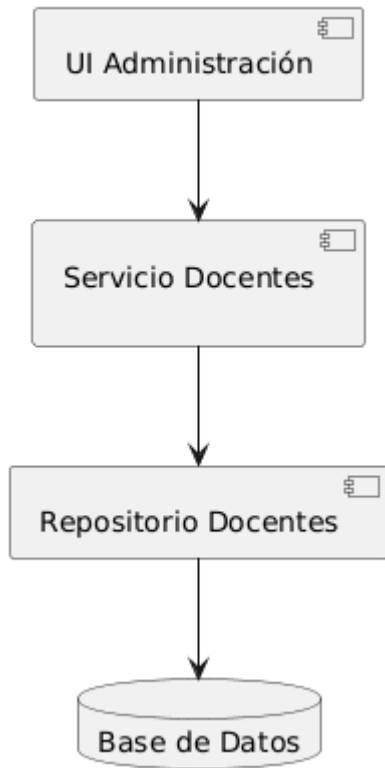
- Gestionar estudiantes

Gestionar Estudiantes



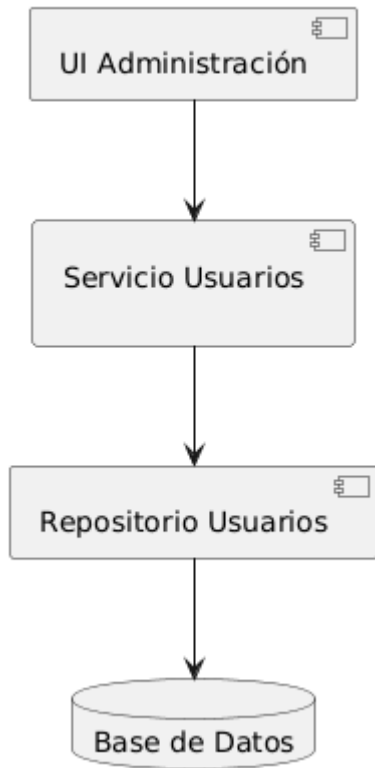
- Gestionar docentes

Gestionar Docentes



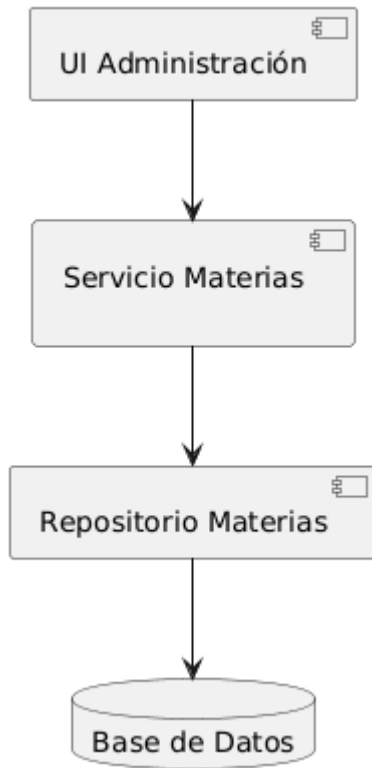
- Gestionar usuarios

Gestionar Usuarios



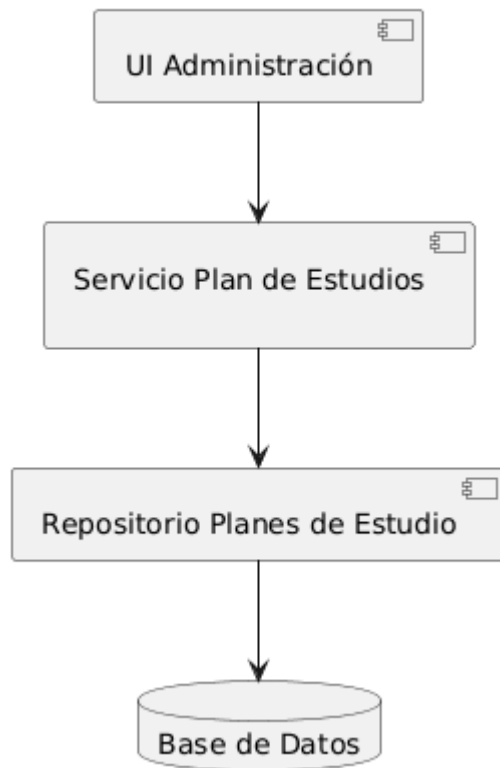
- Gestionar materias

Gestionar Materias



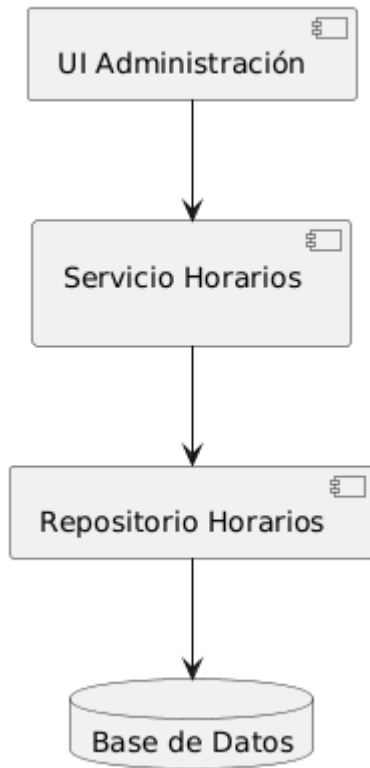
- Gestionar plan de estudios

Gestionar Plan de Estudios



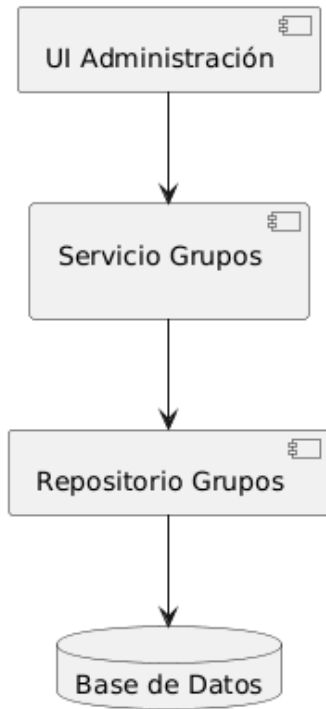
- Gestionar horarios

Gestionar Horarios



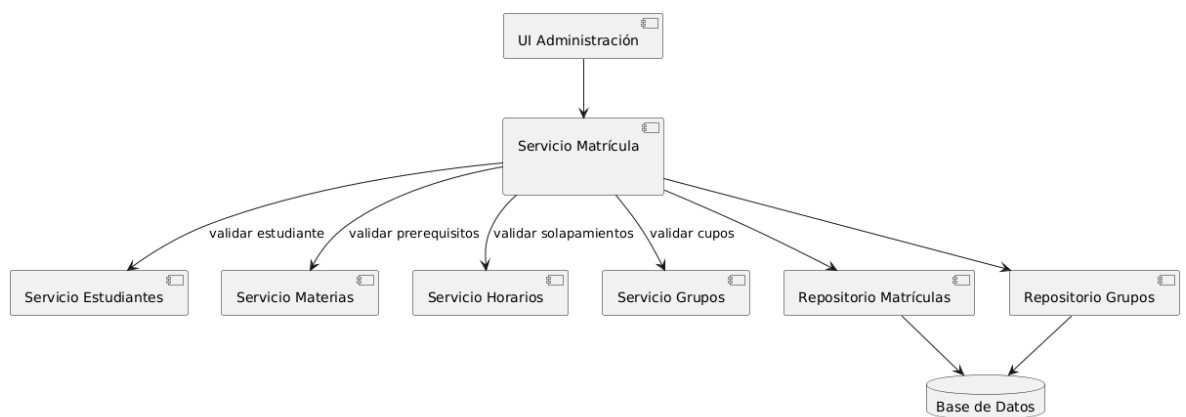
- Gestionar grupos-horarios-profesores

Gestionar Grupos



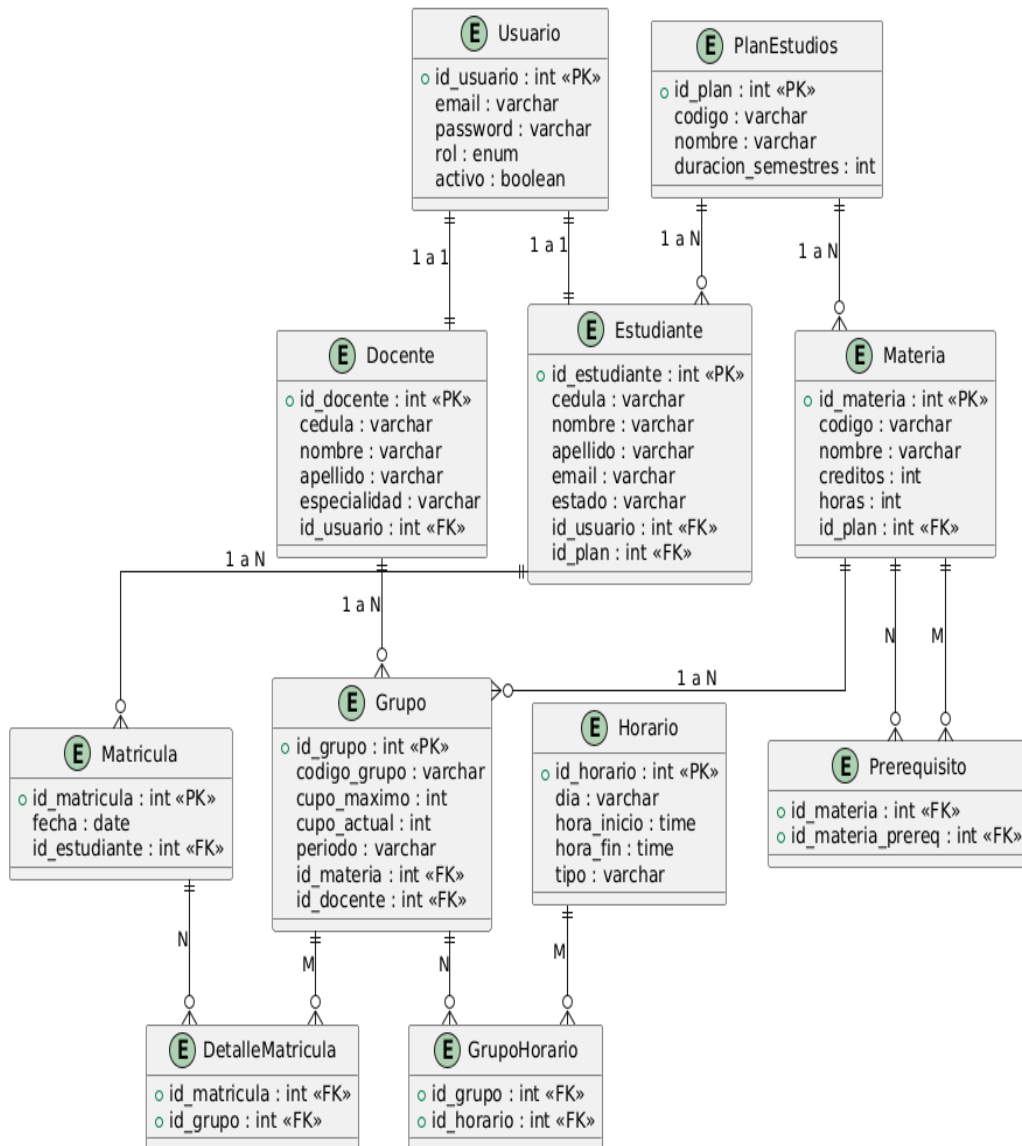
- Matricular estudiante

Matricular Estudiante



Base de datos

Diagrama Entidad-Relación con Cardinalidades - Sistema de Matrículas



Dentro de la base de datos del sistema esta se diseñó bajo un modelo relacional, garantizando la integridad y normalización de la información. La cual está compuesta por entidades principales como usuarios, estudiantes, docentes, materias, grupos, horarios y matrículas, relacionadas mediante claves primarias y foráneas que permiten mantener la consistencia de los datos y soportar las validaciones académicas del sistema.

Diseño del sistema

El diseño del sistema de matrículas se basa en una arquitectura en capas implementada como un monolito modular en Laravel, siguiendo el patrón Modelo–Vista–Controlador reforzado con servicios y repositorios para asegurar una clara separación de responsabilidades. La capa de presentación gestiona la interacción con el usuario mediante vistas Blade y rutas web, mientras que la lógica de negocio se concentra en los controladores y servicios que ejecutan los casos de uso del sistema, como la gestión de estudiantes, asignaturas y procesos de matrícula. La capa de datos se implementa mediante modelos Eloquent, interfaces y repositorios que encapsulan el acceso a la base de datos y garantizan un diseño más flexible y mantenible. El sistema se organiza en módulos funcionales que incluyen autenticación, administración de estudiantes, gestión académica, matrículas y generación de reportes, cada uno compuesto por un conjunto de clases alineadas con la arquitectura. La base de datos se diseñó bajo un modelo relacional con entidades como usuarios, estudiantes, asignaturas, grupos y matrículas, estructuradas mediante claves foráneas que aseguran integridad y normalización. Finalmente, el diseño incorpora mecanismos de seguridad basados en roles, validación de datos en servidor, y flujos de navegación contruidos según los principales casos de uso, con lo cual se obtiene un sistema coherente, escalable y fácil de mantener.

Repositorio y video

Link del repositorio del proyecto: <https://github.com/kazimjesse/Ing-Web-Proyecto-final>

Link del video del proyecto:

https://drive.google.com/file/d/1bdRyH_YMg6sBWExNjk6j37tyZUIAeRhA/view?usp=sharing

Conclusión

Luego del desarrollo del proyecto sistema de matrícula pudimos poner a prueba nuestros conocimientos ya que integramos conceptos de ingeniería web, diseño UML y normalización de bases de datos, demostrando la importancia de estructurar correctamente los casos de uso y aplicar reglas de negocio para garantizar integridad en los procesos. La matriz académica, el control de cupos y los prerrequisitos fueron aspectos clave para simular un sistema académico real y funcional.

Asimismo, el proyecto nos permitió aplicar tecnologías como Laravel bajo el patrón MVC, así como conceptos de arquitectura de software y modelado UML, fortaleciendo nuestras habilidades en el desarrollo de aplicaciones web orientadas a casos reales.

Bibliografía

Administrator, C. (2021, 17 julio). Enrollment System Complete project

documentation | CampCodes. *CampCodes*.

<https://www.campcodes.com/thesis/documentation/enrollment-system-complete-project-documentation/>

School Management System: UML Design & Project Report. (s. f.).

<https://desklib.com/study-documents/software-project-school-system/>

Proyecto de desarrollo de software de la gestión Matrícula. (s. f.). calameo.com.

<https://www.calameo.com/books/0048825364d63e0aef9cb>